RecuVeicol

Gestione del soccorso stradale

Project work · ITS Developer 4.0
Novembre 2026



Team
Vincenzo Minervini · Francesco Valeriano
Giuseppe Cangellosi · Marika Sancilio · Marco Colucci

Cosa dire a voce (60-90 secondi)

Buongiorno. Vi presentiamo RecuVeicol, il project work del corso ITS Developer 4.0. RecuVeicol è un sistema per gestire il soccorso stradale dal primo messaggio del cliente fino alla chiusura dell'intervento. Nel codice e nei documenti troverete anche il nome tecnico FORNITORI, che è il nome del repository: RecuVeicol è il marchio che compare sul sito pubblico. Nei prossimi minuti racconteremo quale problema abbiamo affrontato, come è fatta la soluzione e soprattutto perché abbiamo scelto proprio queste tecnologie. Ogni membro del team presenterà la parte di cui è responsabile. Poi faremo vedere il sistema funzionante, con una missione reale dall'inizio alla fine, su dati dimostrativi. Chiuderemo con i limiti che conosciamo, perché preferiamo dichiararli noi, e con le domande. Tutto ciò che mostriamo esiste nel codice: quando una scelta è ancora da confermare con lei, lo diremo apertamente.

Spiegazione semplice

È la slide di apertura: nome, corso, data e team. RecuVeicol è il marchio, FORNITORI è il nome tecnico del repository e del pacchetto Java.

File principali

`README.md` · `src/main/resources/static/index.html` (marchio RecuVeicol)

Domande del professore

Perché RecuVeicol se il repository si chiama fornitori?

RecuVeicol è il marchio mostrato ai clienti sul sito pubblico; FORNITORI è il nome tecnico nato con il progetto. L'area riservata mostra ancora il logo «fornitori.».

I dati che vediamo sono reali?

No: sono dati sintetici presi dal seed del progetto. Il sistema è una demo e non è pronto per dati reali.

APERTURA

Team e ruoli

Membro	Area	Responsabili
Membro 1	Backend	Vincenzo Minervini
Membro 2	Frontend	Francesco Valeriano, Giuseppe Cangellosi, Marika Sancilio
Membro 3	Chatbot	Marco Colucci
Membro 4	Mappe	Marco Colucci
Membro 5	Notifiche	Marika Sancilio, Francesco Valeriano, Giuseppe Cangellosi

Cinque aree, un contratto API condiviso: ogni componente parla agli altri solo tramite le API.

Cosa dire a voce (60-90 secondi)

Il lavoro è diviso in cinque aree di responsabilità. Vincenzo Minervini è responsabile del backend, cioè del server, del database e del contratto delle API. Francesco Valeriano, Giuseppe Cangellosi e Marika Sancilio curano il frontend: la pagina pubblica, la centrale e l'area autista. Marco Colucci è responsabile del chatbot e delle mappe. Marika, Francesco e Giuseppe si occupano anche delle notifiche al cliente. La divisione non è solo organizzativa: si vede nel codice. Ogni area ha le proprie cartelle e i componenti comunicano soltanto attraverso API descritte in un unico file, `openapi.yaml`. Il bot, per esempio, non legge mai il database. Questo ci ha permesso di lavorare in parallelo: chi consumava un'API poteva rivedere il contratto prima che venisse implementato. Abbiamo lavorato per quattro sprint con uno Scrum adattato, e la tracciabilità dei requisiti è nei documenti.

Spiegazione semplice

Chi fa cosa. Ogni area ha un responsabile e propri file; le aree si parlano tramite API e non toccano le tabelle degli altri.

File principali

`docs/team/README.md` · `docs/team/INTEGRATION_CONTRACTS.md` · `docs/agile/README.md` · `docs/requirements/REQUIREMENTS_TRACEABILITY.md`

Domande del professore

Come vi siete coordinati?

Con il contratto `api/openapi.yaml`: chi propone un cambio apre una issue, il Membro 1 aggiorna il contratto, i consumatori lo rivedono e poi si implementa.

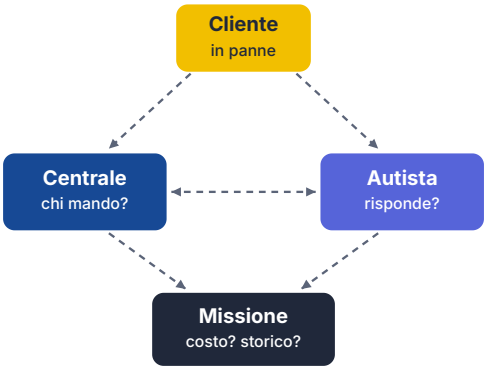
Una persona segue due aree: è un rischio?

Chatbot e mappe sono entrambe di Marco Colucci. I confini sono comunque separati nel codice, e le guide in `docs/team` permettono a un altro membro di riprendere il lavoro.

APERTURA

Il problema

- Chi resta in panne vuole chiedere aiuto subito, anche dal telefono
- La centrale deve trovare un autista libero e vicino
- Se l'autista non risponde, la richiesta non deve fermarsi
- Il cliente vuole sapere chi sta arrivando
- Costi e storico devono restare verificabili



Cosa dire a voce (60-90 secondi)

Partiamo dal problema. Quando un automobilista resta fermo per strada ha bisogno di una cosa semplice: chiedere aiuto dal telefono e sapere chi arriva. Dall'altra parte c'è una centrale operativa che deve capire chi è il cliente, dove si trova e quale autista mandare, scegliendo tra chi è libero e vicino. Poi c'è l'autista, che può rifiutare o non rispondere: in quel caso la richiesta non deve restare ferma, e la centrale deve accorgersene. Infine, a intervento concluso, serve un costo calcolato con regole chiare e uno storico che permetta di ricostruire che cosa è successo. Il problema quindi non è una singola schermata: è coordinare tre persone diverse, su canali diversi, tenendo i dati coerenti anche quando più cose accadono nello stesso momento. Da qui nasce la nostra soluzione.

Spiegazione semplice

Tre attori devono coordinarsi: il cliente chiede, la centrale assegna, l'autista interviene. Le difficoltà sono la mancata risposta, l'avviso al cliente e il costo corretto.

File principali

`docs/requirements/BUR_FORNITORI.md` · `docs/requirements/FUD_FORNITORI.md` · `docs/requirements/BUSINESS_RULES.md`

Domande del professore

Da dove vengono i requisiti?

Dal PDF e dall'Excel del docente. Non sono nel repository: il BUR è una ricostruzione da confermare con lei.

Avete analizzato utenti reali?

No: i dialoghi e i dati vengono dai materiali del corso. Non abbiamo svolto interviste con centrali o autisti.

APERTURA

La soluzione

- **Chat Telegram**: il cliente apre la richiesta con la targa
- **Web centrale**: anagrafiche, missioni, proposta all'autista
- **Web autista**: accetta, vede la mappa, chiude con i km
- **Backend unico**: regole, costi e permessi sul server
- **Notifiche**: il cliente è avvisato in chat



Sito pubblico RecuVeicol: da qui si apre il bot

RecuVeicol Apertura 4 / 40

Cosa dire a voce (60-90 secondi)

La soluzione è un sistema con tre facce e un solo cervello. Il cliente non installa nulla: apre Telegram, scrive la targa e il bot lo guida con poche domande. La richiesta arriva alla centrale, che lavora in una pagina web: gestisce le anagrafiche, vede le missioni e propone l'intervento a un autista, anche con il pulsante «al più vicino». L'autista lavora nella stessa applicazione web, pensata anche per lo smartphone: vede la proposta, il conto alla rovescia e la mappa, accetta o rifiuta, e alla fine chiude la missione indicando i chilometri. Tutte le regole, cioè permessi, tariffe e scadenze, stanno sul server e mai nel browser. Quando l'autista accetta e quando l'intervento è concluso, il cliente riceve un messaggio automatico nella chat. Quello che vedete a destra è il sito pubblico, che spiega il servizio e porta direttamente al bot.

Spiegazione semplice

Il cliente usa Telegram; centrale e autista usano il sito; le decisioni le prende sempre il backend, così nessuno può aggirarle dal browser.

File principali

`README.md` · `src/main/resources/static/index.html` · `chatbot/README.md` · `docs/architecture/TAD_FORNITORI.md`

Domande del professore

Il cliente può chiedere soccorso dal sito?

No: il sito è informativo e rimanda al bot. Le richieste entrano dalla chat o dalla centrale, che può inserirle a mano.

Perché un'unica applicazione web per centrale e autista?

Condividono login e API; il server decide il ruolo e l'interfaccia mostra solo le sezioni di quel ruolo.



Cosa dire a voce (60-90 secondi)

Gli attori sono tre. Il cliente non ha un account: scrive al bot su Telegram e viene riconosciuto dalla targa. Può aprire una richiesta, chiedere lo stato scrivendo STATO e riceve gli avvisi automatici. La centrale corrisponde al ruolo RESPONSABILE: gestisce le anagrafiche, crea e assegna le missioni, vede dashboard e avvisi, corregge l'intervento e può rettificare un costo, ma solo indicando un motivo, che resta registrato. L'autista corrisponde al ruolo AUTISTA: vede soltanto le proprie missioni, ha dieci minuti per rispondere a una proposta, poi invia segnalazioni, chiude con i chilometri e consulta i propri guadagni. Un dettaglio a cui teniamo è la privacy: l'autista vede nome e telefono del cliente solo dopo aver accettato. I ruoli non sono un semplice nascondere pulsanti: è il server a controllarli a ogni richiesta, anche verificando che quella missione appartenga proprio a quell'autista.

Spiegazione semplice

Due ruoli con login (centrale e autista) e un cliente senza login. Il controllo avviene sul server: nascondere un pulsante non basterebbe.

File principali

`src/main/java/it/exprivia/fornitori/config/SecurityConfig.java` · `enums/UserRole.java` · `service/MissionService.java` (metodi `requireRole`, `driverMissionDetail`) · `docs/manuals/MANUALE_UTENTE.md`

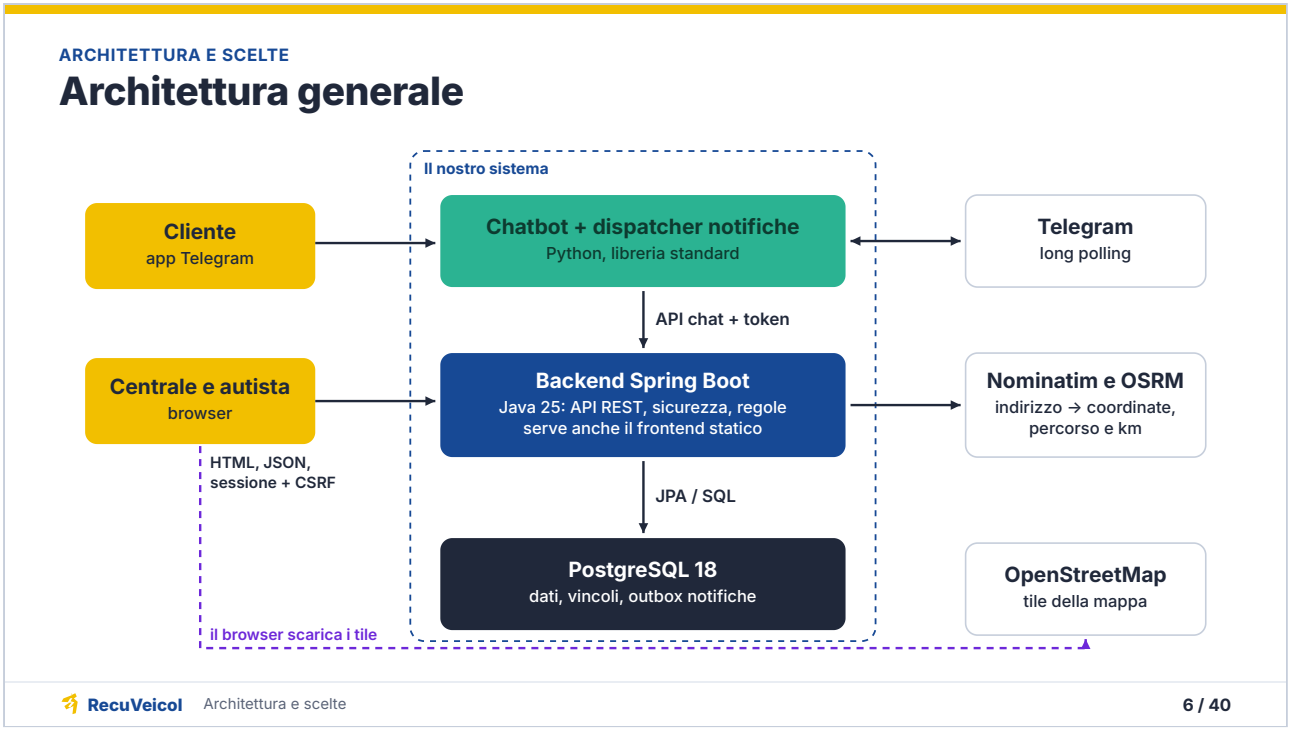
Domande del professore

Un autista può vedere le missioni di un collega?

No: il service verifica che la missione sia assegnata a lui, altrimenti risponde 403. Una vecchia proposta non dà accesso dopo una riassegnazione.

Come si creano nuovi utenti?

Gli account arrivano dal seed. La gestione utenti da interfaccia è fuori ambito: un autista richiederebbe anche un mezzo dedicato.



Cosa dire a voce (60-90 secondi)

Questa è l'architettura. Al centro c'è il backend Spring Boot, l'unico componente che parla con il database PostgreSQL. A sinistra ci sono gli utenti. Centrale e autista usano il browser: il backend serve anche le pagine statiche, quindi tutto parte con un solo processo e le chiamate restano sulla stessa origine, con cookie di sessione e token CSRF. Il cliente usa Telegram. In alto c'è il bot Python: riceve i messaggi da Telegram con il long polling, quindi senza un indirizzo pubblico, e parla con il backend solo tramite le API chat, protette da un token. Lo stesso programma contiene il dispatcher che invia le notifiche. A destra ci sono i servizi esterni: Nominatim trasforma un indirizzo in coordinate, OSRM calcola percorso e chilometri, entrambi chiamati dal backend. I tile della mappa, invece, li scarica direttamente il browser da OpenStreetMap. Il punto chiave è che ogni freccia è un contratto esplicito.

Spiegazione semplice

Un solo server centrale. Bot e browser non toccano mai il database: passano tutti dal backend. I servizi esterni sono usati solo per messaggi e mappe.

File principali

`docs/architecture/TAD_FORNITORI.md` · `docs/diagrams/container.mmd` · `config/SecurityConfig.java` · `chatbot/fornitori_chatbot/http_backend.py`

Alternative, perché scartate, limiti

Alternative: Microservizi separati, oppure bot collegato direttamente al database.

Perché: Il registro decisioni (D-HARD-01) conferma monolite a livelli e API condivise. Il bot senza accesso al DB è il requisito NFR-005: le regole restano in un solo punto.

Limiti e gestione: Se il backend è spento si fermano web e bot, e non è stato provato un deploy reale con HTTPS. È accettabile per una demo; i messaggi in coda non si perdono perché stanno nel database.

Domande del professore

Perché non avete usato i microservizi?

Per una demo con un solo team un monolite a livelli è più semplice da avviare e da testare. Il bot è già separato perché ha un linguaggio e un ciclo di vita propri.

Perché il frontend non è un'app separata?

Spring Boot serve i file statici dalla stessa origine: niente CORS e cookie di sessione più semplici. Si avvia tutto con un comando.

ARCHITETTURA E SCELTE

Tecnologie: cosa e perché

Livello	Tecnologia	Linguaggio	Perché
Frontend	HTML, CSS, Bootstrap 5.3, Leaflet 1.9	JavaScript	Nessuna build: lo serve Spring Boot
Backend	Spring Boot 4.1.2, Security, Data JPA	Java 25	Sicurezza, validazione e dati già integrati
Database	PostgreSQL 18, script numerati	SQL	Vincoli e transazioni proteggono le missioni
Chatbot	Telegram Bot API, long polling	Python 3.10+	Solo libreria standard, nessun indirizzo pubblico
Mappe	Nominatim, OSRM, OpenStreetMap	Java, JS	Open source, senza chiavi a pagamento
Notifiche	Outbox su PostgreSQL + dispatcher	Java, Python	Nessun messaggio perso in silenzio
Build	Maven Wrapper 3.9.9, GitHub Actions, OpenAPI 3.0.3	XML, YAML	Stessa build per tutti, in locale e in CI

RecuVeicol Architettura e scelte 7 / 40

Cosa dire a voce (60-90 secondi)

In questa tabella c'è tutto lo stack, livello per livello. Il frontend è fatto con HTML, CSS e JavaScript, con Bootstrap per il layout e Leaflet per la mappa: non serve alcuna compilazione. Il backend è in Java 25 con Spring Boot, che porta già sicurezza, validazione e accesso ai dati. Il database è PostgreSQL 18, gestito con script SQL numerati. Il chatbot è in Python e usa solo la libreria standard, quindi si avvia senza installare nulla. Per le mappe usiamo Nominatim e OSRM, servizi open source che non richiedono chiavi a pagamento. Le notifiche usano il pattern outbox: una tabella del database fa da coda e il bot le spedisce. Infine, Maven Wrapper garantisce a tutti la stessa versione di Maven, GitHub Actions ripete build e test a ogni modifica e OpenAPI descrive il contratto delle API. Nelle prossime slide giustifichiamo ogni scelta, una per livello, compreso quello che abbiamo scartato.

Spiegazione semplice

Una mappa di tutte le tecnologie. Ricorda la regola: ogni riga ha un motivo pratico legato al progetto, non alla moda del momento.

File principali

`pom.xml` · `.mvn/wrapper/maven-wrapper.properties` · `.github/workflows/backend-ci.yml` · `api/openapi.yaml` · `src/main/resources/static/index.html`

Alternative, perché scartate, limiti

Alternative: L'elenco completo delle alternative è nell'appendice di questa guida.

Perché: Il repository registra lo stack come baseline (D-001) ma non un confronto formale per Java, Spring e PostgreSQL. I motivi in tabella si appoggiano su ciò che il codice usa davvero.

Limiti e gestione: Bootstrap e Leaflet arrivano da CDN: senza Internet l'interfaccia perde stile e mappa. Va quindi controllata la connessione prima della demo.

Domande del professore

Perché così tanti linguaggi?

Java per le regole di business, Python per il bot, JavaScript perché è il linguaggio del browser. Comunicano in JSON, quindi ognuno resta sostituibile.

Ci sono package.json o requirements.txt?

No. Il frontend non ha un package manager e il bot usa solo la libreria standard: le uniche dipendenze sono nel `pom.xml`.

Perché Java 25?

È il target LTS condiviso dal team (H-005). `java.version` e `maven.compiler.release` sono impostati a 25.



Cosa dire a voce (60-90 secondi)

Il backend è organizzato su tre livelli, ed è la cosa più importante da capire. Il controller riceve la richiesta HTTP e valida il JSON, senza logica. Il service contiene le regole e apre le transazioni: qui si decide se una missione può essere accettata. Il repository, con le entità JPA, parla con PostgreSQL. In mezzo ci sono i DTO, oggetti che definiscono ciò che entra ed esce, così la password o le entità non vengono mai esposte. Spring Boot ci dà tre cose che avremmo dovuto riscrivere: Spring Security per sessione, CSRF e ruoli; la validazione con annotazioni; JPA e Hibernate per transazioni e lock pessimistici, fondamentali quando due persone agiscono sulla stessa missione. Maven gestisce build e dipendenze in un unico file. Con il Maven Wrapper chiunque usa la stessa versione, la 3.9.9, senza installarla, e lo stesso comando vale anche nella pipeline.

Spiegazione semplice

Tre strati, ognuno con un compito. Se cambia una regola tocchi il service; se cambia una query tocchi il repository. Spring evita di riscrivere sicurezza e accesso ai dati.

File principali

`pom.xml` · `src/main/java/it/exprivia/fornitori/controller/` · `service/MissionService.java` · `repository/` · `entity/` · `dto/` · `mvnw`

Alternative, perché scartate, limiti

Alternative: Node.js con Express; Python con Django; Gradle al posto di Maven.

Perché: Node e Django richiederebbero di scegliere e configurare a parte sicurezza e ORM. Nel repo non c'è un confronto formale: il motivo documentato è il target condiviso Java 25 (H-005, D-001), il resto deriva da ciò che il codice usa.

Limiti e gestione: `MissionService` è grande (715 righe) ed è debito tecnico noto. Le liste non sono paginate (V-23): con i dati della demo non è un problema.

Domande del professore

Perché non avete usato Node.js?

Spring offre già sessione, CSRF, validazione e JPA in modo coerente e con lock pessimistici semplici da usare. Con Node avremmo dovuto assemblare più librerie.

Che differenza c'è tra entity e DTO?

L'entity rispecchia la tabella; il DTO è ciò che viaggia in JSON. Separandoli non esponiamo per errore hash o colonne interne.

Cos'è il Maven Wrapper?

Lo script `mvnw` scarica la versione di Maven indicata nel repository, così tutti compilano allo stesso modo.



Cosa dire a voce (60-90 secondi)

I dati di questo sistema sono fortemente collegati: una missione ha un cliente, un autista, forse un'officina, poi proposte, note ed eventi. Un database relazionale è la scelta naturale, perché le regole possono stare anche nel database. Le chiavi esterne impediscono una missione senza cliente; i vincoli CHECK impediscono uno stato inesistente o un costo negativo; le transazioni fanno sì che la missione accettata e la notifica vengano salvate insieme o non vengano salvate affatto. Di PostgreSQL usiamo in particolare gli indici parziali: un indice unico garantisce che un autista abbia al massimo una missione attiva, anche se due richieste arrivano nello stesso istante. E usiamo FOR UPDATE SKIP LOCKED per la coda delle notifiche. Lo schema nasce da script SQL numerati, da applicare in ordine: sono versionati e rileggibili. Hibernate non modifica mai le tabelle, si limita a controllarle all'avvio: se manca uno script, l'applicazione non parte.

Spiegazione semplice

Il database è l'ultima difesa: anche con un bug nel codice, un autista non può avere due missioni attive. Gli script numerati sono la storia delle modifiche allo schema.

File principali

`database/01_schema.sql` · `database/02_seed.sql` · `database/08_cleanup_and_delivery.sql` · `database/09_integrity_and_indexes.sql` · `database/10_night_rate_22.sql` · `src/main/resources/application.properties` · `docs/architecture/DATABASE_DESIGN.md`

Alternative, perché scartate, limiti

Alternative: MySQL; un database NoSQL; schema generato da Hibernate.

Perché: MySQL non ha indici parziali con WHERE, usati da `uq_driver_active_live` e `uq_one_open_proposal`. NoSQL non offre lo stesso controllo su relazioni e vincoli. Lo schema generato nasconderebbe le modifiche: `ddl-auto=validate` le rende esplicite. Il repo non contiene un confronto formale: il motivo deriva dal codice.

Limiti e gestione: `fornitori_app` è proprietario delle tabelle (V-13, aperta): in produzione andrebbero separati ruolo delle migrazioni e ruolo applicativo. Gli script 03 e 04 non esistono: la numerazione ha un salto.

Domande del professore

Perché non avete usato MySQL?

Il nostro vincolo «un autista, una missione attiva» è un indice parziale con WHERE, che MySQL non supporta in questa forma. Andrebbe spostato nel codice o simulato.

Perché non usare Flyway o Liquibase?

Non sono nel `pom.xml`: gli script si applicano a mano con `psql`. È un miglioramento possibile; per la demo il controllo `validate` basta a segnalare gli script mancanti.

Come fate a non rovinare i dati storici?

Le 30 missioni del seed hanno `is_fixture=TRUE`: lo scheduler e le azioni live le ignorano e non vengono ricalcolate.

ARCHITETTURA E SCELTE

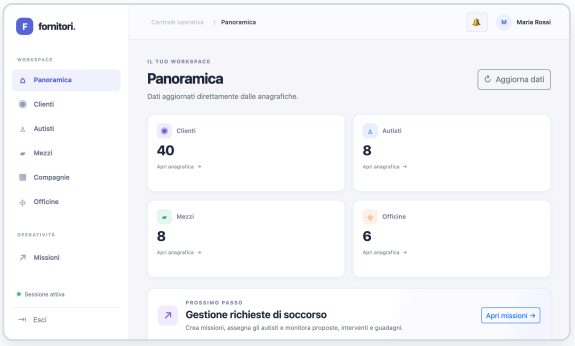
Frontend: JavaScript senza framework e Leaflet

Perché HTML, CSS e JavaScript

- Nessuna compilazione né Node.js
- Servito da Spring Boot: stessa origine, sessione semplice
- Bootstrap per un layout pronto e responsive

Perché Leaflet

- Libreria open source per mappe
- Disegna il percorso GeoJSON di OSRM
- Nessuna chiave a pagamento



Panoramica della centrale, schermata reale

RecuVeicol Architettura e scelte 10 / 40

Cosa dire a voce (60-90 secondi)

Per il frontend abbiamo usato HTML, CSS e JavaScript puro, con Bootstrap e Leaflet. Non c'è un framework come React e non c'è una fase di compilazione: i file stanno in `src/main/resources/static` e li serve direttamente Spring Boot. Questo ha tre vantaggi pratici. Primo, non serve installare Node.js: per avviare il progetto basta il comando del backend. Secondo, pagine e API hanno la stessa origine, quindi cookie di sessione e token CSRF funzionano senza configurare CORS. Terzo, Bootstrap ci dà un layout responsive, utile perché l'autista lavora dal telefono. Per la mappa usiamo Leaflet, una libreria open source che riceve direttamente la geometria GeoJSON calcolata da OSRM e la disegna sui tile di OpenStreetMap. Il prezzo di questa semplicità è un unico file `app.js` di quasi 1.400 righe: non è il massimo, ma è tutto in un posto e non dipende da build.

Spiegazione semplice

Il browser riceve file pronti e li esegue così come sono. Niente passaggi intermedi: meno strumenti, avvio più semplice. In cambio il codice è meno modulare.

File principali

`src/main/resources/static/index.html` · `static/app.js` · `static/styles.css` · `static/public-site.css` · `docs/team/MEMBRO_2_FRONTEND.md`

Alternative, perché scartate, limiti

Alternative: React, Angular o Vue; Google Maps al posto di Leaflet.

Perché: Un framework aggiungerebbe Node.js e una build, che il manuale tecnico esclude esplicitamente. Il repo documenta la scelta (D-010) ma non il confronto con React: il motivo è ricostruito dalla struttura del codice.

Limiti e gestione: `app.js` è un file unico; gli aggiornamenti usano un controllo ogni 15 secondi; Bootstrap e Leaflet arrivano da CDN, con hash di integrità. Servono quindi Internet e un ridisegno manuale delle viste.

Domande del professore

Perché non avete usato React?

Le schermate sono poche e guidate dal server. React avrebbe richiesto Node e una build; qui basta avviare il backend. Con più schermate lo rivaluteremmo.

Come si evita l'XSS senza framework?

I dati dinamici passano da una funzione di escaping (`safe`), e la Content-Security-Policy limita gli script alle fonti ammesse.

Il sito funziona senza Internet?

No: Bootstrap, Leaflet e i tile arrivano da servizi esterni. In locale si vede la pagina, ma senza stili e mappa.

ARCHITETTURA E SCELTE

Chatbot e notifiche: Python, Telegram, outbox

Perché Python
- Solo libreria standard
- Avvio immediato, senza installazioni
- Dialogo a stati facile da leggere

Telegram, non WhatsApp
- API gratuita e aperta
- Nessuna approvazione Meta
- Adatta alla demo: niente server pubblico

Perché l'outbox
- Evento salvato con la missione
- Nessun messaggio perso in silenzio
- Nessun doppio invio tra dispatcher



Caso limite noto: se il bot si ferma tra invio ed esito, quel messaggio può ripartire una volta.

Cosa dire a voce (60-90 secondi)

Tre scelte da giustificare. Il bot è in Python e usa solo la libreria standard: si avvia senza installare dipendenze e il dialogo si legge come una macchina a stati. Per il canale abbiamo scelto Telegram invece di WhatsApp. L'API di Telegram è gratuita e aperta, non serve l'approvazione di Meta e, con il long polling, non serve neppure un indirizzo pubblico: è ideale per una demo. Il canale WhatsApp resta fuori ambito. La terza scelta è il pattern outbox. Se il backend inviasse il messaggio mentre salva la missione, un guasto di Telegram bloccherebbe il lavoro, o la missione verrebbe annullata dopo che il messaggio è partito. Noi invece scriviamo l'evento in una tabella, nella stessa transazione della missione. Poi il bot riserva le righe e le invia. Così una notifica non si perde in silenzio e due dispatcher non ricevono la stessa riga. Resta un caso limite dichiarato: un crash tra invio ed esito può ripetere il messaggio.

Spiegazione semplice

L'outbox è una lista di cose da spedire dentro il database. Prima si scrive «da inviare», poi qualcun altro spedisce e segna l'esito. Se il bot è spento, la lista aspetta.

File principali

`chatbot/fornitori_chatbot/engine.py` · `chatbot/fornitori_chatbot/telegram_channel.py` · `chatbot/fornitori_chatbot/notifications.py` · `service/MissionService.java` (`addOutboxEvent`) · `entity/NotificationOutbox.java` · `docs/management/DECISION_LOG.md` (D-008, D-009, D-015, H-009)

Alternative, perché scartate, limiti

Alternative: Bot in Java dentro il backend o con la libreria python-telegram-bot; WhatsApp Business API; invio diretto senza outbox.

Perché: WhatsApp richiede approvazione Meta e un webhook pubblico. Il long polling è documentato in H-009. I motivi su API gratuita e approvazione Meta vengono dalla scelta del team: nel codice non ci sono prove. La scelta di Python non è motivata nei documenti: è il fatto che il bot non abbia dipendenze a sostenerla.

Limiti e gestione: Consegna «almeno una volta»: Telegram non offre chiavi anti-duplicati (V-14, accettata per la demo). Dopo 5 tentativi una riga va in FAILED: non è inviata, ma resta registrata con l'errore. Il PDF del docente può prevedere WhatsApp: va confermato.

Domande del professore

Perché non avete usato WhatsApp?

Richiede account Business, approvazione Meta e un webhook pubblico, cioè costi e tempi fuori dalla demo. Il motore del bot è però separato dal canale: basta un adattatore.

Garantite che non ci siano duplicati?

Tra dispatcher sì, grazie a SKIP LOCKED. Un doppio invio resta possibile solo se il bot si ferma tra invio ed esito. Lo dichiariamo.

Se Telegram è giù, la missione si blocca?

No: la missione viene salvata comunque. La riga resta in coda, riprova dopo 30 secondi, fino a 5 tentativi.



Cosa dire a voce (60-90 secondi)

Come stanno insieme i pezzi. Abbiamo separato i componenti e li abbiamo collegati con API REST che scambiano JSON. Qui vedete un esempio reale: il bot manda sessione e targa, il backend risponde se la targa è riconosciuta, con nome e compagnia. Tutte le API sono descritte in `openapi.yaml`, che è la fonte ufficiale: contiene 38 operazioni, di cui 34 implementate e quattro solo progettate, cioè gestione utenti e webhook, che sono fuori ambito. Un test confronta il contratto con il codice. Il vantaggio della separazione si vede nel bot: ha un'interfaccia verso il backend con due implementazioni, una HTTP reale e una finta, così si può provare da console senza server. Per le mappe usiamo Nominatim e OSRM, open source e gratuiti, senza chiavi a pagamento. Li chiama sempre il backend, con timeout e limiti, così il browser non parla mai con loro e una loro caduta produce un errore chiaro.

Spiegazione semplice

REST significa: richieste HTTP con dati JSON. Il contratto è l'accordo scritto tra chi offre l'API e chi la usa; se cambia, tutti lo devono rivedere.

File principali

`api/openapi.yaml` · `docs/team/INTEGRATION_CONTRACTS.md` · `docs/architecture/API_SPECIFICATION.md` · `chatbot/fornitori_chatbot/backend.py` · `chatbot/fornitori_chatbot/fake_backend.py` · `service/NominatimGeocodingClient.java` · `service/OsrmRoutingClient.java` · `src/test/java/.../ContractAndProviderTests.java`

Alternative, perché scartate, limiti

Alternative: GraphQL o gRPC per le API; Google Maps o Mapbox per le mappe.

Perché: REST con JSON è leggibile da Java, Python e browser senza librerie extra. H-010 documenta la scelta: servizi pubblici senza chiavi e costi nel perimetro didattico.

Limiti e gestione: Nominatim e OSRM pubblici non garantiscono disponibilità (R-04) e ricevono gli indirizzi: adatto alla demo, da rivalutare per dati reali. Errori espliciti e chiusura sempre possibile.

Domande del professore

Perché non avete usato Google Maps?

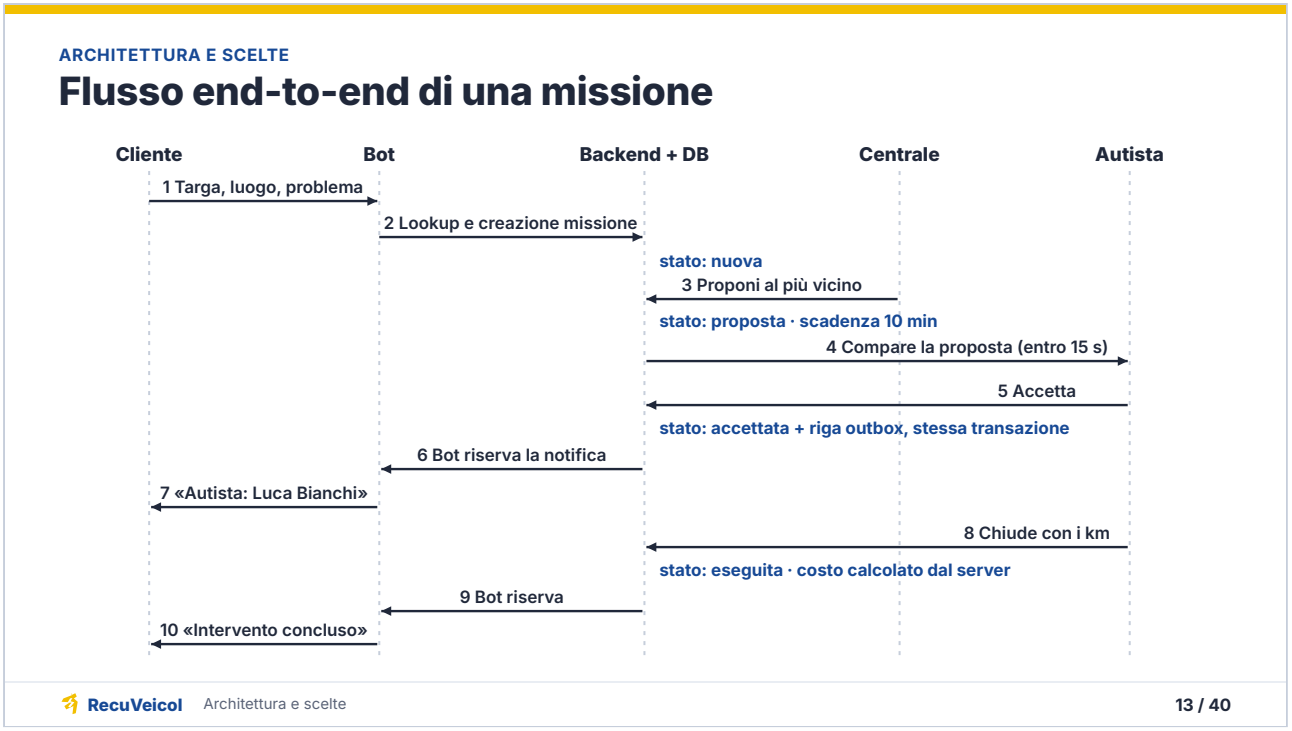
Richiede una chiave con fatturazione. Nominatim e OSRM bastano per una demo e sono open source; in produzione useremmo un provider con garanzie.

Che cosa succede se il contratto e il codice divergono?

I test in `ContractAndProviderTests` confrontano i due. Il processo prevede di aggiornare OpenAPI prima di implementare.

Perché il browser non chiama direttamente OSRM?

Il backend applica permessi e limiti di frequenza e salva i km stimati. La CSP ammette connessioni solo verso la stessa origine.



Cosa dire a voce (60-90 secondi)

Ecco una missione dall'inizio alla fine, ed è esattamente ciò che vedrete nella demo. Il cliente scrive al bot targa, luogo e problema. Il bot controlla la targa e crea la missione tramite l'API: lo stato è nuova. La centrale la vede e preme «Proponi al più vicino»: il backend sceglie l'autista libero più vicino, lo stato diventa proposta e parte una scadenza di dieci minuti. Entro quindici secondi la proposta compare nell'area dell'autista. L'autista accetta: nella stessa transazione la missione diventa accettata e nasce la riga di notifica. Il bot, che controlla ogni cinque secondi, la riserva e la invia: il cliente legge il nome dell'autista. Alla fine l'autista chiude indicando i chilometri, il server calcola il costo, e parte l'ultima notifica. Osservate che il bot e l'interfaccia non si parlano mai direttamente: passa tutto dal backend, e lo stato vive nel database.

Spiegazione semplice

Segui i numeri dall'alto in basso. Gli stati della missione stanno sempre nel backend; bot e pagine web mostrano solo una vista di quello stato.

File principali

`docs/diagrams/mission_states.mmd` · `docs/diagrams/chat_notifications_sequence.mmd` · `service/ChatService.java` · `service/NearestDriverService.java` · `service/MissionService.java` · `chatbot/fornitori_chatbot/notifications.py`

Domande del professore

Se l'autista non risponde?

Uno scheduler controlla ogni 30 secondi le scadenze: la missione diventa scaduta e la centrale riceve un avviso. Il controllo vale anche nell'istante della risposta.

E se rifiuta?

Lo stato diventa non accettata, parte l'avviso e la centrale ripropone. «Al più vicino» esclude chi ha già risposto; lo storico delle proposte resta.

La proposta arriva in tempo reale?

Quasi: la pagina controlla ogni 15 secondi. Non usiamo WebSocket, per tenere semplice il client.



Cosa dire a voce (60-90 secondi)

Ora entriamo nelle singole parti, partendo dal cuore del sistema. Sono Vincenzo Minervini e sono responsabile del backend e del database. In questa sezione vedremo prima le API, cioè che cosa il backend offre agli altri componenti. Poi la sicurezza: come funzionano login, token CSRF e ruoli. Poi la logica delle missioni: gli stati, la tariffa notturna, le scadenze e l'assegnazione al più vicino. Chiuderemo con lo schema del database, semplificato, e con una slide che spiega perché il codice è organizzato così. Il messaggio di fondo è uno: il server non si fida mai del client. Costi, permessi e scadenze sono calcolati e controllati solo qui. Anche se qualcuno modificasse la pagina nel browser, non potrebbe accettare una missione di un altro autista né cambiare il proprio guadagno. Tutto questo è coperto da 139 test automatici su un database isolato.

Spiegazione semplice

Il backend è il programma Java che custodisce regole e dati. Tutti gli altri componenti gli chiedono le cose tramite API.

File principali

`src/main/java/it/exprivia/fornitori/` · `database/` · `api/openapi.yaml` · `docs/team/MEMBRO_1_BACKEND.md` · `docs/security/SECURITY_TEST_REPORT.md`

Domande del professore

Avete dei test?

Sì: 139 test Java su un database isolato e 6 test del chatbot, tutti superati. Girano solo su database `fornitori_e2e*`, perché cancellano i dati live.

Quanto è grande il backend?

6 controller, 14 repository, 13 entità e i service: `MissionService` è il più grande, con 715 righe.

BACKEND E DATABASE

API: chi può chiamare cosa

Area	Op.	A cosa serve	Chi accede
Autenticazione	4	Login, profilo, logout, CSRF	Pubblico, poi sessione
Anagrafiche	6	Clienti, autisti, mezzi, compagnie, officine	Centrale
Missioni e dashboard	16	Ciclo missione, note, costi, storico	Centrale o autista assegnato
Assegnazione	2	Proposta al più vicino, avvisi	Centrale
Chat e notifiche	5	Targa, missione, stato, invio	Bot, con token di servizio
Mappe	1	Percorso e km stimati	Centrale o autista assegnato

Errori sempre uguali: `{ status, error, message }`

Cosa dire a voce (60-90 secondi)

Il backend espone 34 operazioni REST, raggruppate in sei aree. L'autenticazione ne ha quattro: il token CSRF, il login, il profilo e il logout. Le anagrafiche ne hanno sei e sono riservate alla centrale. La parte più ampia è quella delle missioni: creazione, proposta, accettazione, rifiuto, chiusura, rettifica del costo, note, dashboard e storico dell'autista. Le API di assegnazione offrono la proposta automatica al più vicino e gli avvisi alla centrale. Le API chat sono usate solo dal bot, con un token di servizio. Infine c'è l'unica API delle mappe. Tutte rispondono in JSON, e gli errori hanno sempre la stessa forma: lo stato HTTP, un codice stabile e, quando serve, un messaggio per l'utente. Così il frontend può mostrare per esempio «Targa già presente» senza conoscere i dettagli interni. In caso di errore inatteso rispondiamo con un 500 generico e un riferimento da cercare nel log, senza mai mostrare eccezioni o SQL.

Spiegazione semplice

Ogni riga è un gruppo di «sportelli». La colonna a destra dice chi può andarci. Il formato d'errore uguale per tutti semplifica il lavoro di chi usa l'API.

File principali

`api/openapi.yaml` · `controller/MissionController.java` · `controller/RegistryController.java` · `controller/ChatController.java` · `controller/AssignmentController.java` · `common/ApiExceptionHandler.java` · `common/ApiError.java`

Alternative, perché scartate, limiti

Alternative: Pagine generate dal server (Thymeleaf) o GraphQL.

Perché: Un'unica API REST serve sia il browser sia il bot, in JSON. Thymeleaf non sarebbe servito al bot. Nel repo non c'è un confronto formale: il motivo discende dall'avere due consumatori diversi.

Limiti e gestione: Liste non paginate (V-23). La GET delle mappe può scrivere sul database (V-24): è protetta da CSRF, ma cambiarla romperebbe il contratto.

Domande del professore

Perché i verbi in inglese, come /accept, in un'API italiana?

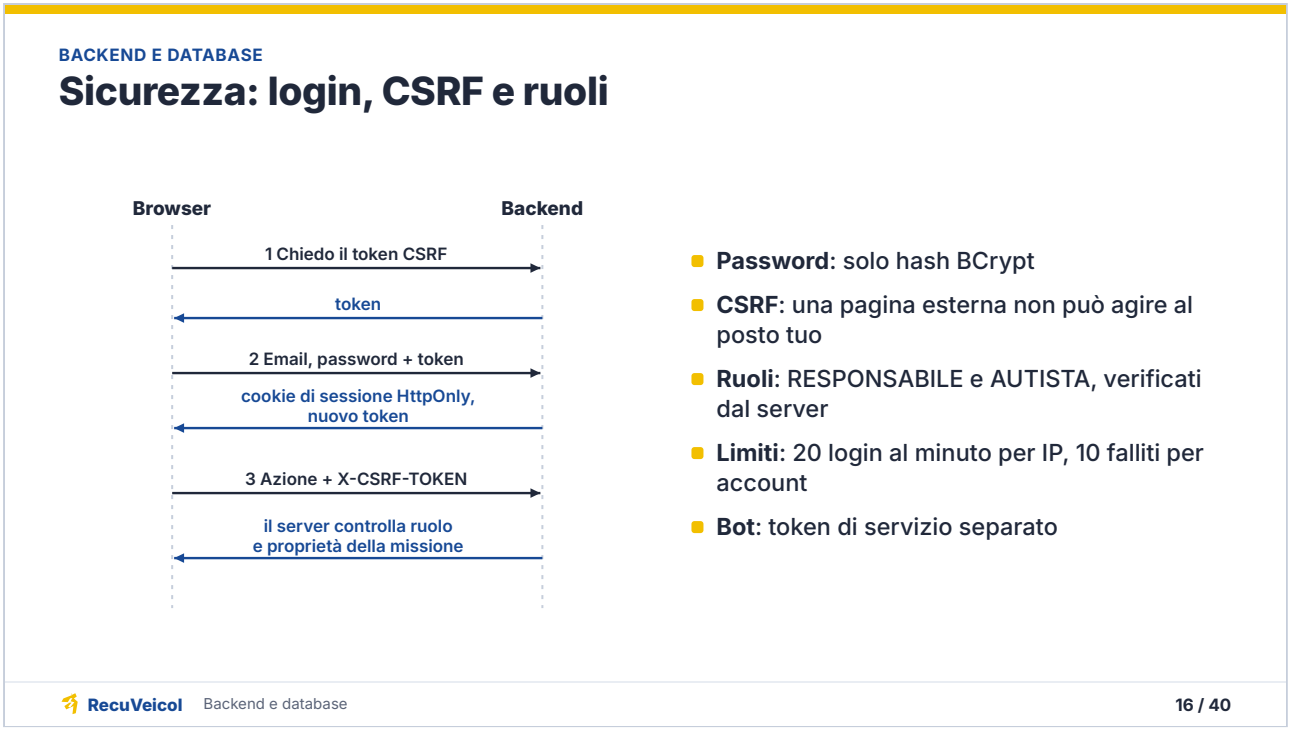
È una disomogeneità del contratto: `missioni` è italiano, `accept`, `reject` e `complete` sono inglesi. Non cambia il funzionamento, ma non ha una motivazione documentata.

Come fa il bot a non passare dal login?

Ha una catena di sicurezza separata su `/api/v1/chat/**`, stateless, con un token bearer confrontato a tempo costante.

Cosa succede se un'API riceve JSON enorme?

`RequestBodyLimitFilter` rifiuta i corpi oltre 64 KiB con 413, prima di arrivare al controller.



Cosa dire a voce (60-90 secondi)

Vediamo la sicurezza con il login. Il browser chiede prima un token CSRF, poi invia email, password e token. Se le credenziali sono corrette, riceve un cookie di sessione HttpOnly, che JavaScript non può leggere, e un nuovo token, perché al login cambiano sia l'identificativo di sessione sia il CSRF. Ogni azione successiva allega il token in un'intestazione. Il CSRF serve a questo: senza token, una pagina esterna aperta nello stesso browser non può far partire un'azione a tuo nome, anche se il cookie viene inviato. Le password sono salvate solo come hash BCrypt. I ruoli sono controllati su due livelli: Spring Security filtra per percorso, e il service verifica che la missione sia davvero di quell'autista. Ci sono limiti ai tentativi: 20 login al minuto per indirizzo IP e 10 falliti per account. Il bot non usa la sessione: ha una catena di sicurezza separata, con un token di servizio. Tutto questo è stato verificato con test specifici.

Spiegazione semplice

La sessione dice «chi sei», il token CSRF prova «sei stato davvero tu a premere», il ruolo dice «cosa puoi fare». Sono tre controlli diversi.

File principali

`config/SecurityConfig.java` · `controller/AuthController.java` · `config/ChatServiceTokenFilter.java` · `service/RequestRateLimiter.java` · `docs/architecture/SECURITY_DESIGN.md` · `docs/security/VULNERABILITY_REGISTER.md`

Alternative, perché scartate, limiti

Alternative: JWT nel browser invece della sessione.

Perché: H-004 documenta la scelta: sessione server-side, login JSON, cookie HttpOnly, BCrypt e CSRF. Un JWT salvato nel browser sarebbe leggibile da JavaScript e difficile da revocare; un logout con la sessione la invalida subito.

Limiti e gestione: Il cookie `Secure` è attivo solo nel profilo `prod` e HTTPS non è stato provato su un server reale (V-15). I limiti sono in memoria, per singola istanza (V-22).

Domande del professore

Perché non avete usato JWT?

Il client è sulla stessa origine, quindi la sessione con cookie è più semplice e si revoca subito. Il token del bot è opaco, non un JWT.

Perché serve il CSRF se c'è il cookie HttpOnly?

HttpOnly protegge dalla lettura del cookie via script. Il CSRF protegge da richieste inviate da altri siti, in cui il cookie partirebbe da solo.

Perché la GET delle mappe richiede il CSRF?

Può salvare coordinate e km stimati: non è una lettura pura, quindi la proteggiamo come una modifica.



Cosa dire a voce (60-90 secondi)

La missione è una macchina a stati. Nasce nuova, la centrale la propone, poi può essere accettata, rifiutata o può scadere; nei due ultimi casi si può riproporre, e alla fine viene eseguita. Ogni passaggio prende un lock sulla riga, così due azioni contemporanee non si sovrappongono. Le scadenze: una proposta vale dieci minuti, valore nella tabella `pricing_config`. Uno scheduler controlla ogni trenta secondi, e il controllo viene ripetuto quando l'autista risponde. La tariffa: cinquanta euro di uscita più un euro e venti al chilometro, con il venticinque per cento in più nella fascia notturna. Vale l'ora di accettazione, nel fuso di Roma. Attenzione: abbiamo scelto la fascia dalle 22 alle 8, mentre il PDF indica dalle 20 alle 8, e lo vogliamo confermare con lei. Il più vicino: si calcola la distanza in linea d'aria tra il luogo del sinistro e la base dei mezzi, tra gli autisti liberi che non hanno già risposto.

Spiegazione semplice

Una missione può trovarsi in un solo stato alla volta e passa solo per le frecce disegnate. Esempio reale della demo: 15,23 km di giorno = 50 + 1,20 × 15,23 = €68,28; la stessa missione di notte = €85,35.

File principali

`service/MissionService.java` · `service/PricingService.java` · `service/MissionExpiryScheduler.java` · `service/NearestDriverService.java` · `service/GeoDistance.java` · `database/10_night_rate_22.sql` · `docs/diagrams/mission_states.mmd` · `docs/requirements/BUSINESS_RULES.md`

Alternative, perché scartate, limiti

Alternative: Calcolo del costo nel client; `double` invece di `BigDecimal`; più vicino per strada con OSRM.

Perché: Costo solo sul server, perché è un dato da non poter manipolare. `BigDecimal` con HALF_UP evita errori di arrotondamento (H-003). La linea d'aria non richiede chiamate esterne per ogni autista.

Limiti e gestione: La linea d'aria può scegliere un autista non ottimale per strada. Le 22:00 e il riferimento all'ora di accettazione vanno confermati (D-HARD-03). Nello storico 6 missioni su 30 sono notturne: 3 di queste lo sono solo con la vecchia fascia e non vengono ricalcolate. La rettifica del costo resta possibile, con motivo obbligatorio.

Domande del professore

Perché le 22:00 se il PDF dice 20:00?

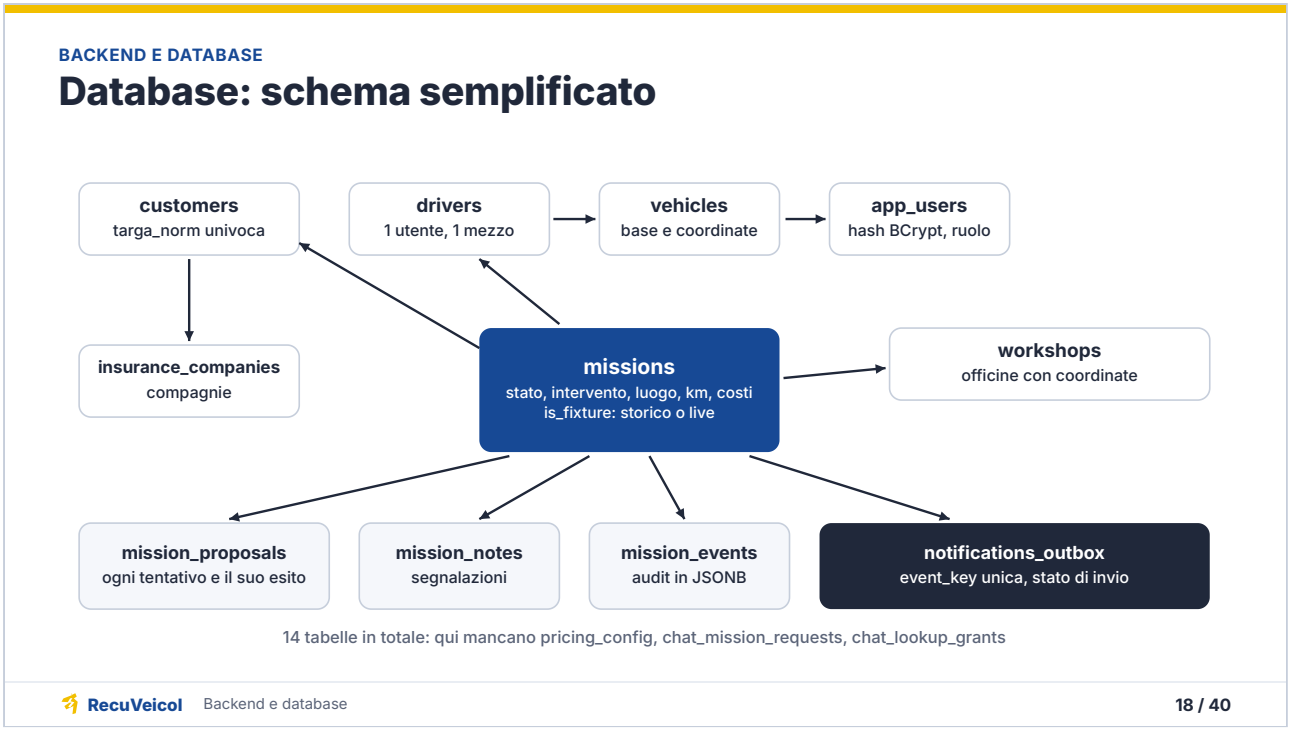
È una scelta del team, documentata in D-HARD-03 e da confermare con il docente. Il valore è in `pricing_config`: per cambiarlo basta aggiornare una riga.

Perché non avete usato la distanza stradale per il più vicino?

Servirebbe una chiamata a OSRM per ogni autista, verso un servizio pubblico con limiti. Per scegliere bastano le coordinate; i km da pagare invece sono quelli effettivi.

Che succede se due centrali propongono lo stesso autista?

Lock pessimistico su missione e autista, e indice unico parziale. Il secondo riceve 409; «al più vicino» passa al successivo.



Cosa dire a voce (60-90 secondi)

Questo è lo schema semplificato. Al centro c'è la tabella `missions`, che collega un cliente, eventualmente un autista e un'officina, e contiene stato, luogo, chilometri e costi. Il cliente ha una targa normalizzata univoca e una compagnia assicurativa. L'autista è legato a un solo utente e a un solo mezzo; il mezzo contiene le coordinate della base, che servono per scegliere il più vicino. Sotto vedete quattro tabelle figlie. `mission_proposals` conserva ogni tentativo, così sappiamo chi ha rifiutato e quando. `mission_notes` contiene le segnalazioni dell'autista. `mission_events` è l'audit, con i dettagli in JSON. `notifications_outbox` è la coda delle notifiche. In tutto le tabelle sono 14: mancano dal disegno la configurazione tariffaria e le due tabelle del chatbot. Una colonna da ricordare è `is_fixture`: separa le 30 missioni storiche fornite dal docente da quelle operative, così lo storico non viene mai alterato.

Spiegazione semplice

Una missione è il «foglio» centrale; le altre tabelle sono le schede collegate. Non si cancella la storia: proposte ed eventi si aggiungono.

File principali

`database/01_schema.sql` · `database/05_chat_requests.sql` · `database/07_backend_hardening.sql` · `docs/architecture/DATABASE_DESIGN.md` · `docs/diagrams/database_er.mmd` · `entity/Mission.java`

Alternative, perché scartate, limiti

Alternative: Storico in un'unica colonna JSON della missione; tabella separata per le missioni storiche.

Perché: `mission_proposals` e `mission_events` separate restano interrogabili e rendono esplicito l'audit. Il flag `is_fixture` è documentato in H-001: non alterare il riepilogo storico. Importi in NUMERIC e istanti in TIMESTAMPTZ.

Limiti e gestione: Importi del seed: 16 eseguite, totale €1.509,71. Alcune incoerenze del foglio Excel restano nelle righe `is_fixture` (R-08), isolate dalle regole live. Excel originale non presente nel repository.

Domande del professore

Perché NUMERIC e non FLOAT per il costo?

FLOAT accumula errori di arrotondamento. NUMERIC(10,2) è esatto, e in Java corrisponde a BigDecimal.

Perché ci sono targa e targa_norm?

`targa` conserva come è scritta, `targa_norm` è solo maiuscole e cifre e serve per la ricerca: «pw 660 sw» e «PW660SW» sono la stessa targa.

Perché uno stesso autista non può avere due missioni?

Lo vieta `uq_driver_active_live`, indice unico sulle sole missioni live in stato proposta o accettata.



Cosa dire a voce (60-90 secondi)

Chiudo la parte backend con il perché. L'organizzazione dei file segue i livelli: chi cerca una regola sa di dover aprire `service`; chi cerca una query apre `repository`. I DTO stanno in un pacchetto a sé, così il formato pubblico non è legato alle tabelle. Gli script SQL sono numerati per ricostruire il database da zero, in ordine. Sul fronte dei pattern, i livelli separano le responsabilità. Il lock pessimistico evita che due azioni si sovrappongano sulla stessa missione, cosa fondamentale in un sistema dove due persone possono agire nello stesso secondo. Il tempo non viene letto direttamente: viene iniettato un Clock, e questo permette ai test di verificare una scadenza senza aspettare dieci minuti. Sui formati: JSON nelle API con errori uniformi, NUMERIC e BigDecimal per i soldi, TIMESTAMPTZ per gli istanti e JSONB per i dettagli dell'audit, che cambiano da evento a evento. Nulla di esotico: scelte standard e verificabili.

Spiegazione semplice

Tutto è al suo posto, per chi legge il codice. I soldi sono calcolati con precisione esatta; il tempo si può simulare nei test.

File principali

`src/main/java/it/exprivia/fornitori/` (cartelle) · `config/TimeConfiguration.java` · `repository/MissionRepository.java` (`lockLiveById`) · `entity/MissionEvent.java` · `src/test/java/.../MissionLifecycleTests.java`

Alternative, perché scartate, limiti

Alternative: Lock ottimistico con versione; un'architettura esagonale; date in stringa.

Perché: D-HARD-01 conferma i livelli e correzioni mirate. Con il lock ottimistico il secondo utente otterrebbe comunque un errore dopo il tentativo; il pessimistico è più semplice da ragionare per transizioni brevi. Quest'ultimo confronto non è documentato: è una lettura del codice.

Limiti e gestione: `MissionService` concentra molte responsabilità ed è debito tecnico noto. Le mappe non sono transazionali di proposito, per non tenere aperta una transazione durante una chiamata esterna.

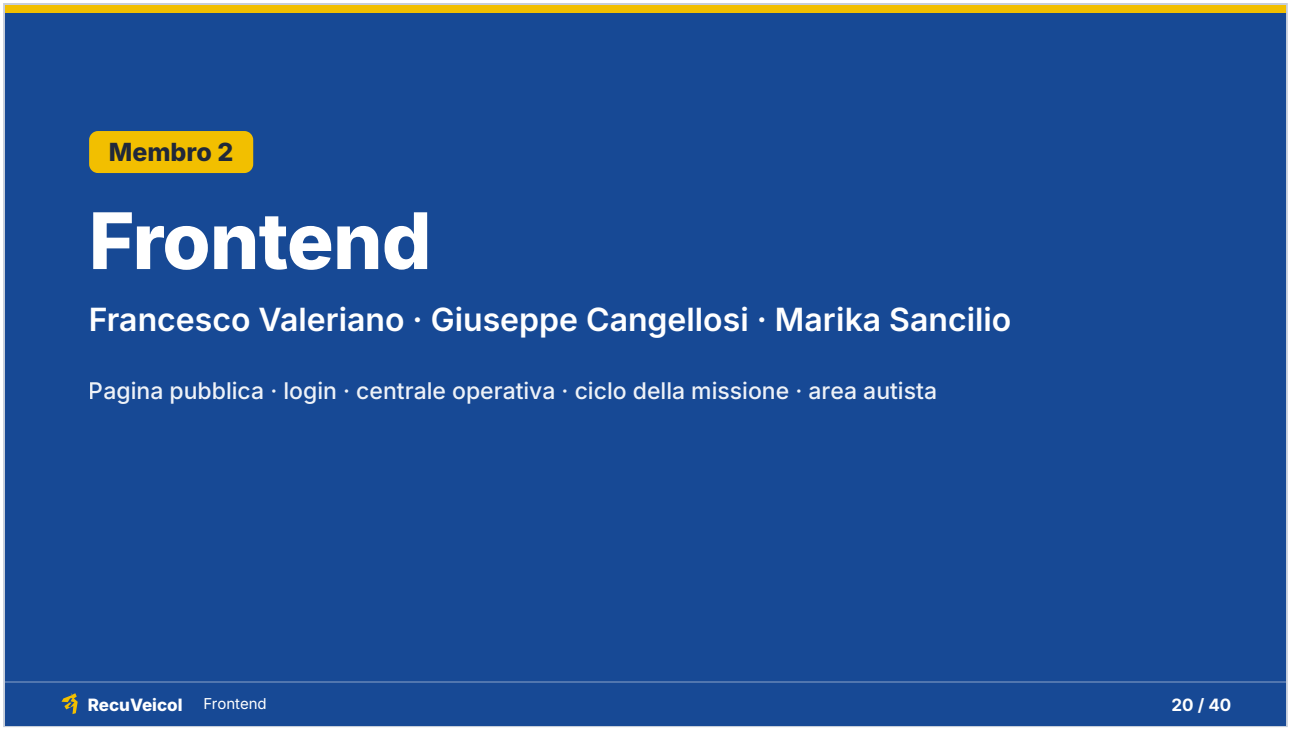
Domande del professore

Perché MapService non è @Transactional?

Perché chiama servizi esterni che rispondono in secondi: una transazione aperta bloccherebbe risorse. I risultati si salvano in transazioni brevi.

Cosa cambieresti del backend?

Dividerei `MissionService` per casi d'uso e aggiungerei la paginazione alle liste: sono i debiti già registrati in TAD e V-23.



Cosa dire a voce (60-90 secondi)

Passiamo ora a ciò che gli utenti vedono. Noi siamo Francesco Valeriano, Giuseppe Cangellosi e Marika Sancilio, responsabili del frontend. L'applicazione web è una sola e offre tre esperienze: la pagina pubblica, che spiega il servizio e porta al bot; la centrale operativa, per chi coordina; l'area autista, pensata per lo smartphone. Tutto sta in una cartella di file statici, con un solo `index.html` e un solo `app.js`: la vista cambia in base a ciò che l'utente sceglie e al ruolo restituito dal server. Le prossime slide mostrano schermate reali catturate dal sistema in funzione, su dati dimostrativi. Vedrete prima la panoramica e le anagrafiche, poi il ciclo della missione con la proposta e gli avvisi, e infine l'area autista dal telefono. Una regola ha guidato tutto il lavoro: il frontend non calcola e non decide nulla, mostra e chiede soltanto. Ogni dato arriva dalle API e nessuna mappa o informazione è simulata.

Spiegazione semplice

Il frontend è ciò che si vede nel browser. Le decisioni le prende sempre il backend; qui si disegnano i risultati e si raccolgono i comandi dell'utente.

File principali

`src/main/resources/static/` · `docs/team/MEMBRO_2_FRONTEND.md` · `docs/manuals/MANUALE_UTENTE.md`

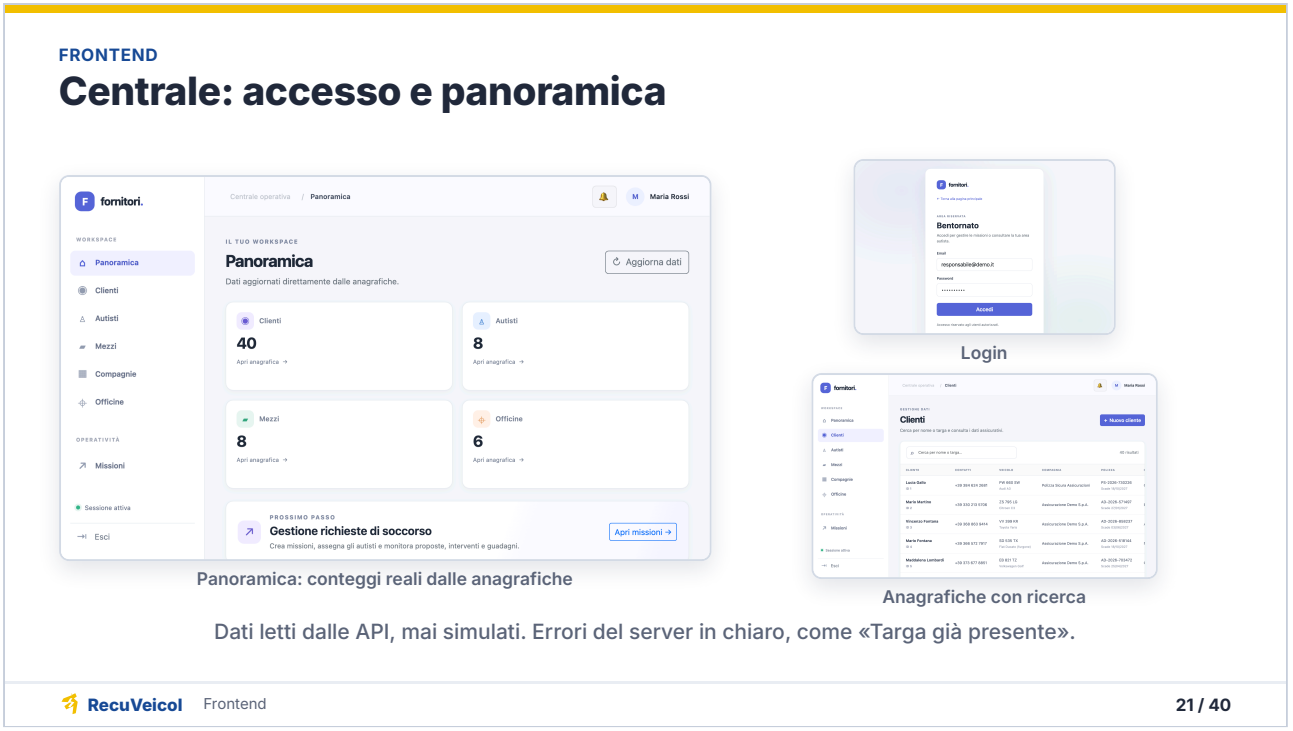
Domande del professore

È una single page application?

Di fatto sì: una sola pagina HTML con viste mostrate e nascoste da JavaScript, ma senza framework.

L'interfaccia è accessibile?

L'HTML usa etichette e attributi ARIA, ma l'accessibilità non è stata verificata in modo formale: il rapporto dei test la segna come non verificata.



Cosa dire a voce (60-90 secondi)

Partiamo dall'accesso. Il login usa email e password; dopo l'autenticazione il server restituisce il ruolo e l'interfaccia mostra solo le sezioni di quel ruolo. Nella schermata del login vedete il logo «fornitori.», il nome tecnico, che l'area riservata conserva ancora. La panoramica della centrale riassume i conteggi: 40 clienti, 8 autisti, 8 mezzi e 6 officine. Sono numeri reali, letti dalle anagrafiche attraverso le API. Da qui si entra nelle singole anagrafiche: clienti, autisti, mezzi, compagnie e officine. In basso a destra c'è la lista dei clienti, con la ricerca per nome o targa e l'inserimento di un nuovo cliente tramite una finestra guidata. Se il server rifiuta il dato, per esempio perché una targa esiste già, la pagina mostra il suo messaggio, senza inventare un testo. Ogni lista ha un indicatore di caricamento e uno stato vuoto, così l'utente capisce sempre che cosa sta succedendo. Il layout è responsive: si adatta da solo allo schermo.

Spiegazione semplice

Tutto ciò che è a schermo arriva dal server. Se il server dice no, l'utente legge il motivo del server.

File principali

`static/index.html` (`loginView`, `dashboardPage`, `registryPage`) · `static/app.js` (`loadRegistries`, `request`) · `service/RegistryService.java` · `static/styles.css`

Alternative, perché scartate, limiti

Alternative: Tabelle con un componente dedicato (DataTables) o un framework CSS diverso.

Perché: Bootstrap copre griglia, modali e tabelle senza altre dipendenze. Il repo non motiva Bootstrap rispetto ad altri framework CSS: è documentato solo come presente (TAD).

Limiti e gestione: Liste non paginate. Il marchio resta «fornitori.» nell'area riservata e RecuVeicol nel sito pubblico: da uniformare.

Domande del professore

Perché vedo «fornitori» nel login?

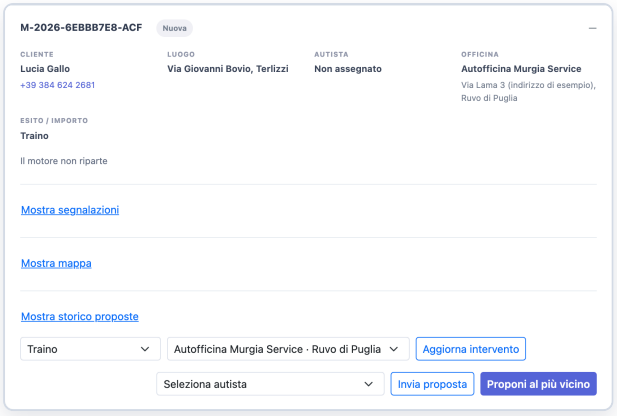
Il marchio RecuVeicol è sul sito pubblico; l'area riservata mantiene il nome tecnico del progetto. Andrà allineato.

Come si cerca un cliente?

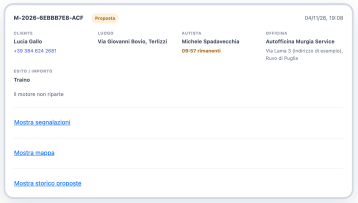
La ricerca chiama `GET /api/v1/clienti?search=` e tratta il testo come tale: `%` e `_` non restituiscono tutti i clienti (V-16, corretta).

FRONTEND

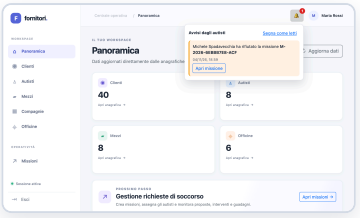
Centrale: il ciclo della missione



1 · Missione nuova dal bot: si propone al più vicino



2 · Proposta: conto alla rovescia di 10 minuti



3 · Rifiuto: la campanella avvisa la centrale

Riproposta con storico. Intervento e officina correggibili. Costo rettificabile con motivo.

RecuVeicol Frontend 22 / 40

Cosa dire a voce (60-90 secondi)

Ecco il lavoro quotidiano della centrale. La scheda a sinistra è una missione appena arrivata dal bot: mostra cliente, luogo, tipo di intervento e officina assegnata. Qui la centrale può scegliere un autista dall'elenco oppure premere «Proponi al più vicino», lasciando decidere il sistema. Può anche correggere tipo di intervento e officina, se quanto indicato dal cliente non è adatto. Appena proposta, in alto a destra, compare lo stato con il conto alla rovescia dei dieci minuti. Se l'autista rifiuta o la proposta scade, la campanella segnala l'avviso: il controllo avviene ogni quindici secondi. Dall'avviso si torna alla scheda e si ripropone: «Al più vicino» esclude chi ha già risposto, e ogni tentativo resta nello storico delle proposte. Quando la missione è eseguita, compare il campo per correggere il costo. La correzione richiede un motivo, che resta registrato, e il costo calcolato non viene perso.

Spiegazione semplice

La centrale non deve ricordare nulla: la scheda mostra lo stato e le sole azioni possibili in quello stato. Il sistema rifiuta comunque quelle non ammesse.

File principali

`static/app.js` (`renderMissionList`, `loadAlerts`, `updateCountdowns`) · `controller/MissionController.java` · `controller/AssignmentController.java` · `service/CentreAlertService.java`

Alternative, perché scartate, limiti

Alternative: WebSocket o Server-Sent Events per aggiornamenti in tempo reale.

Perché: Il polling ogni 15 secondi è documentato e basta per un processo in cui si risponde in minuti. Il repo non documenta il confronto con i WebSocket: il motivo è ricostruito (meno infrastruttura lato client e server).

Limiti e gestione: Avvisi fino a 15 secondi in ritardo. Lo stato «letto» degli avvisi è salvato solo nel browser di chi guarda, quindi non è condiviso tra operatori.

Domande del professore

Perché non WebSocket?

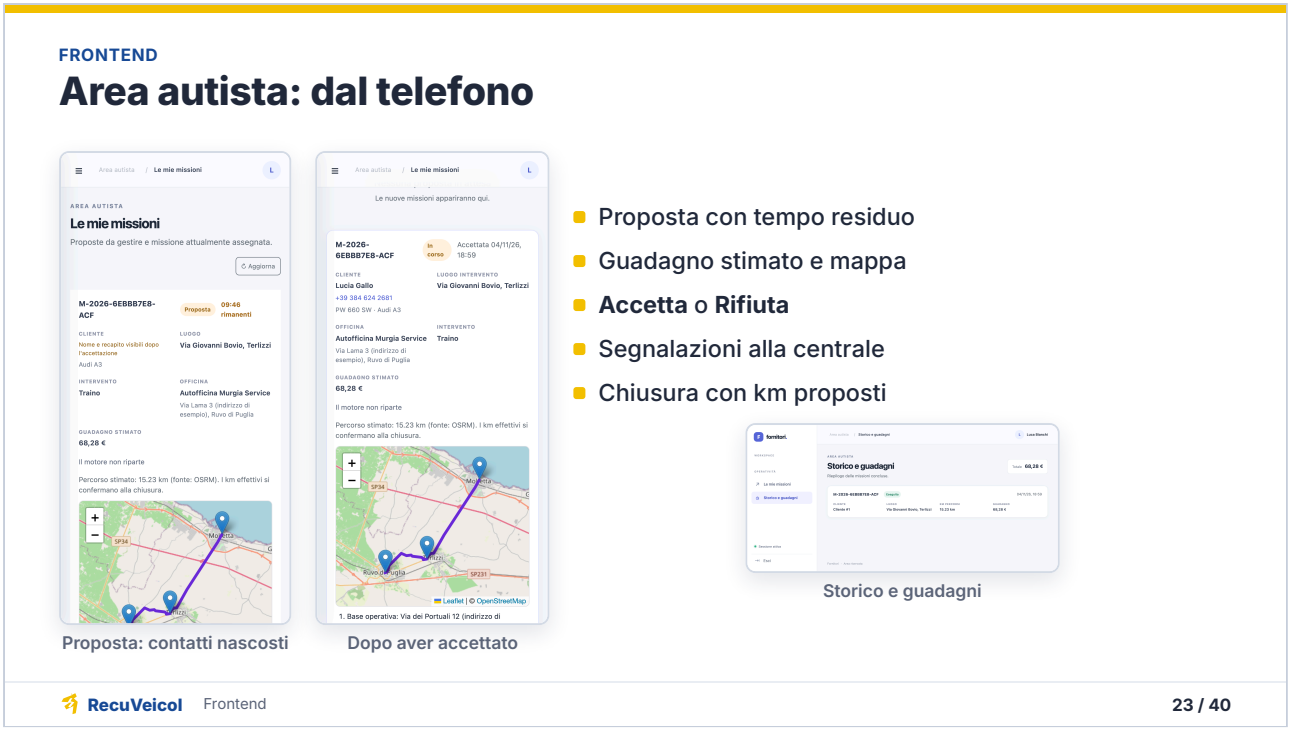
Bastano aggiornamenti ogni 15 secondi e il polling non richiede altro server né gestione delle connessioni.

Può la centrale creare una missione da sola?

Sì, con «Nuova missione»: seleziona cliente, intervento, officina se è un traino, canale, luogo e problema. Quelle missioni però non generano notifiche, perché la chat del cliente non è nota.

Il canale «WhatsApp» nel modulo invia messaggi?

No: è solo un metadato di origine. WhatsApp non è integrato.



Cosa dire a voce (60-90 secondi)

L'area autista è pensata per chi è in strada, quindi per lo smartphone. Le nuove proposte compaiono da sole: la pagina controlla ogni quindici secondi e si ridisegna solo se qualcosa è cambiato. Nella prima schermata vedete una proposta: il tempo residuo, il luogo, l'officina, il guadagno stimato e la mappa. Nome e telefono del cliente non ci sono ancora: compare la scritta «visibili dopo l'accettazione». Questo non è un effetto grafico: è il server a non inviare quei dati finché l'autista non accetta. Nella seconda schermata la missione è stata accettata e compaiono i contatti del cliente. L'autista può inviare una segnalazione alla centrale e, a fine intervento, chiudere la missione. Il campo dei chilometri è precompilato con la stima del percorso, ma l'autista inserisce quelli effettivi. Se il servizio mappe non risponde, può inserirli comunque. In basso, lo storico mostra le missioni concluse e il totale dei guadagni.

Spiegazione semplice

L'autista vede solo le sue missioni. I dati personali del cliente arrivano dal server solo dopo l'accettazione.

File principali

`static/app.js` (`driverActiveCard`, `pollDriverMissions`, `loadDriverHistory`) · `service/MissionService.java` (`driverMissionDetail`, `complete`) · `controller/MissionController.java` (`/autista/*`) · `static/styles.css`

Alternative, perché scartate, limiti

Alternative: Un'app nativa Android/iOS.

Perché: Una pagina responsive non richiede installazioni né pubblicazione negli store, e riusa lo stesso login. Il repo documenta l'ottimizzazione per smartphone, non un confronto con le app native.

Limiti e gestione: Senza app non ci sono notifiche push: l'autista deve tenere aperta la pagina. La mappa richiede connessione.

Domande del professore

Come sa l'autista di avere una proposta?

La pagina la controlla ogni 15 secondi. Non ci sono notifiche push: è un limite noto.

Può chiudere la missione con km inventati?

Il server accetta solo valori non negativi, ma i km sono dichiarati dall'autista. La centrale vede anche i km stimati e può rettificare il costo con un motivo.

Può mandare note prima di accettare?

No: il server risponde 409 (V-12, corretta).



Cosa dire a voce (60-90 secondi)

Perché abbiamo fatto il frontend così? Per i file: tutto sta in `static`, con un solo HTML e un solo JavaScript, e due fogli di stile separati, uno per l'area riservata e uno per il sito pubblico. È una struttura che si capisce in pochi minuti e che si avvia con il backend. Il pattern è semplice: un oggetto di stato in memoria, riempito dalle API, e funzioni che lo disegnano. Le viste si mostrano e si nascondono via JavaScript, senza ricaricare la pagina. Gli aggiornamenti avvengono con un controllo ogni quindici secondi, e nell'area autista si ridisegna solo se qualcosa è cambiato, per non far perdere il testo che si sta scrivendo. Il formato dei dati è il JSON delle API, sulla stessa origine, con cookie di sessione e il token CSRF nell'intestazione. Tutti i dati dinamici passano da una funzione di escaping, perché un nome cliente non deve poter diventare codice eseguito nella pagina.

Spiegazione semplice

Pochi file, un unico modo di lavorare: chiedi dati, salvali, ridisegna. L'escaping trasforma i caratteri speciali in testo innocuo.

File principali

`static/app.js` (`state`, `safe`, `request`) · `static/index.html` · `static/styles.css` · `static/public-site.css` · `src/test/java/.../ContractAndProviderTests.java`

Alternative, perché scartate, limiti

Alternative: Dividere il JavaScript in moduli; usare un framework reattivo.

Perché: Senza build, più file significherebbero più richieste e più ordine da gestire. Il repo non documenta una motivazione esplicita per il file unico: è una conseguenza della scelta senza build.

Limiti e gestione: `app.js` di 1.391 righe è poco modulare: una crescita delle schermate renderebbe opportuno dividerlo o introdurre un framework.

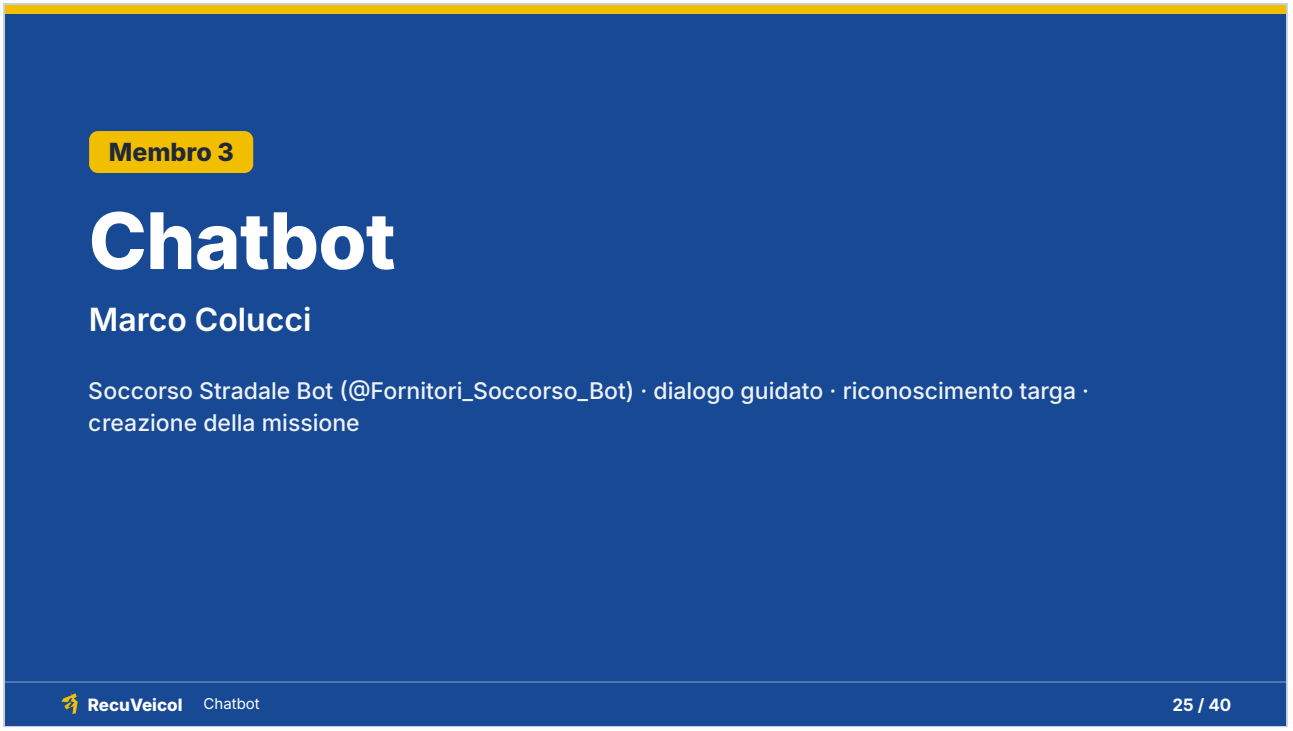
Domande del professore

Dove si mette un nuovo controllo?

In `index.html` il markup, in `app.js` l'azione che chiama l'API. Le regole restano nel backend.

Cosa fa `safe`?

Sostituisce i caratteri HTML speciali prima di inserire i dati in pagina: è la difesa contro l'XSS, insieme alla Content-Security-Policy.



Cosa dire a voce (60-90 secondi)

Sono Marco Colucci e ho curato il chatbot. Su Telegram il bot si chiama Soccorso Stradale Bot, con username Fornitori Soccorso Bot. È il punto di contatto con il cliente, che non deve installare nulla oltre a Telegram e non deve registrarsi. Il dialogo è guidato: poche domande precise, una alla volta, come previsto dai dialoghi forniti nel corso. Abbiamo scelto un flusso deterministico, senza intelligenza artificiale: il cliente è in difficoltà e deve ricevere risposte prevedibili. Vedremo il dialogo vero e proprio, poi come il bot riconosce la targa e crea la missione. Sottolineo fin da ora un limite che riprenderemo: la sola targa identifica il cliente. È in linea con i dialoghi del docente, ma non è adatto a dati reali, ed è la voce aperta V-01 del registro di sicurezza. Il bot non vede mai il database: fa tutto chiamando il backend, con un token di servizio.

Spiegazione semplice

Il bot è un programma che chiede le informazioni nell'ordine giusto e le passa al backend. Non decide le regole e non conserva i dati delle missioni.

File principali

`chatbot/README.md` · `chatbot/fornitori_chatbot/` · `docs/team/MEMBRO_3_CHATBOT.md` · `docs/management/DECISION_LOG.md` (D-005, D-007)

Domande del professore

Perché non c'è l'intelligenza artificiale?

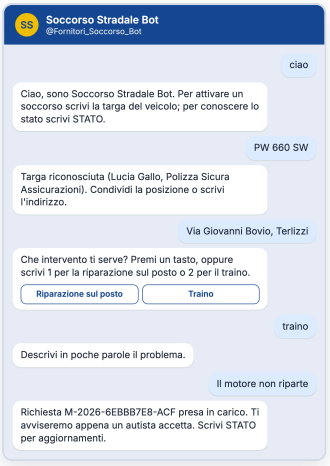
Il dialogo 10 con Gemini è facoltativo e non è implementato. Un flusso a parole chiave è prevedibile e verificabile (D-005).

Serve Telegram per provarlo?

No: con `python3 -m fornitori_chatbot` si usa la console e un backend finto; con `--http` si usa il backend reale.

CHATBOT

Il dialogo con il cliente



Testi reali, prodotti dal bot. Telegram simulato nella grafica.

- Una domanda alla volta
- Posizione condivisa o indirizzo scritto
- Riparazione o traino, con i tasti
- **AIUTO**, **ANNULLA**, **STATO** in ogni momento
- Linguaggio semplice, mai stati interni



Dialogo inattivo per 30 minuti: scade. 3 risposte non valide: ricomincia.

Cosa dire a voce (60-90 secondi)

Questo è un dialogo reale: i testi li ha prodotti il nostro bot collegato a un backend di prova. Solo la grafica di Telegram è simulata. Il cliente scrive un saluto e il bot si presenta. Scrive la targa e il bot risponde con nome e compagnia, poi chiede dove si trova: si può condividere la posizione oppure scrivere via e città. Poi il bot chiede che intervento serve, con due tasti: riparazione sul posto o traino. Infine chiede di descrivere il problema, crea la richiesta e mostra il codice. In ogni momento il cliente può scrivere AIUTO, ANNULLA o STATO. Tecnicamente è una macchina a stati, con quattro stati in `engine.py`, e tutti i testi stanno in `messages.py`, separati dalla logica. Il dialogo scade dopo trenta minuti di inattività e, dopo tre risposte non valide, riparte. STATO non mostra mai gli stati interni: traduce in una frase comprensibile, per esempio «stiamo cercando un autista disponibile».

Spiegazione semplice

Il bot ricorda a che punto è ogni chat e interpreta il messaggio in base a quel punto. Così la stessa parola può voler dire cose diverse a seconda del momento.

File principali

`chatbot/fornitori_chatbot/engine.py` · `chatbot/fornitori_chatbot/messages.py` · `chatbot/fornitori_chatbot/telegram_channel.py` · `docs/diagrams/chatbot_sequence.mmd`

Alternative, perché scartate, limiti

Alternative: Linguaggio naturale con AI; dialogo salvato nel database; libreria python-telegram-bot.

Perché: Il foglio DialoghiChatbot prevede un flusso guidato. La tabella `chat_sessions` è stata rimossa dallo script 08: i dialoghi stanno in memoria. La sola libreria standard elimina dipendenze esterne.

Limiti e gestione: Un riavvio del bot azzera i dialoghi in corso (R-07), non le missioni. I dialoghi 2-4 hanno un passo in più del foglio docente: la scelta dell'intervento.

Domande del professore

Se il bot si riavvia durante un dialogo?

Il cliente ricomincia. Le missioni già create e il loro stato restano nel backend e si possono consultare con STATO.

Perché ci sono tasti se è una chat?

Riducono gli errori nella scelta: premere un tasto equivale a scrivere il suo valore. Si può comunque scrivere 1 o 2.

Questo dialogo è stato provato su Telegram?

Lo screenshot è simulato. Il repository segna la prova reale da telefono come da ripetere: prima della demo va eseguita.



Cosa dire a voce (60-90 secondi)

Vediamo che cosa succede dietro le quinte. Il cliente scrive la targa, anche in minuscolo o con spazi: il bot la normalizza e la manda al backend. Il backend cerca il cliente e controlla che la polizza non sia scaduta. Se è tutto a posto, restituisce nome e compagnia e crea un'autorizzazione: per quindici minuti, per quella sessione e quel cliente, si può aprire una sola richiesta. Se scade durante il dialogo, il bot rifà da solo il controllo una volta. Raccolti luogo, intervento e problema, il bot chiama la creazione della missione con un requestId. Questo è il nostro antiduplicato: se la rete cade e il bot ritenta, l'identificativo è lo stesso e non nasce una seconda missione. Se viene riusato con dati diversi, il backend risponde 409. Per il traino il backend sceglie l'officina più vicina, trasformando l'indirizzo in coordinate. Contro i tentativi a caso, dopo cinque targhe non trovate il backend blocca la sessione.

Spiegazione semplice

La targa apre una porta per poco tempo e per una sola richiesta. Il requestId è come il numero di una raccomandata: se la rispedisci, non diventa una seconda lettera.

File principali

`service/ChatService.java` · `service/ChatLookupLimiter.java` · `service/NearestWorkshopResolver.java` · `repository/ChatLookupGrantRepository.java` · `chatbot/fornitori_chatbot/engine.py` (`normalize_plate`) · `chatbot/fornitori_chatbot/http_backend.py` · `database/07_backend_hardening.sql`

Alternative, perché scartate, limiti

Alternative: Verifica con un secondo dato o con un codice SMS; il cliente sceglie l'officina.

Perché: D-007: come nei dialoghi del docente, basta la targa. Il PDF chiede solo di verificare che la targa corrisponda a un assicurato. H-008: il cliente in panne non sa quale officina scegliere, quindi la assegna il backend.

Limiti e gestione: V-01 aperta: chi conosce una targa vede nome e compagnia e può creare una richiesta. L'audit propone di non mostrare più quei dati e aggiungere le ultime 4 cifre della polizza o un codice SMS. Officina scelta in linea d'aria, correggibile dalla centrale. I limiti sono in memoria.

Domande del professore

Chiunque può vedere nome e compagnia con una targa?

Sì, è il limite V-01, dichiarato e ad alta gravità. È adatto solo a dati sintetici. Per dati reali serve un secondo fattore: la soluzione è già descritta nell'audit.

Perché non chiedete il numero di telefono?

Telegram non lo comunica al bot. Per questo lo stato si cerca per chat e non per numero.

Cosa impedisce di provare migliaia di targhe?

5 targhe non trovate per sessione, 10 ricerche al minuto per sessione, 100 al minuto globali. Una targa trovata non azzera il contatore (V-03, corretta).



Cosa dire a voce (60-90 secondi)

Perché il bot è fatto così? I file separano tre preoccupazioni. `engine.py` contiene la logica del dialogo; `messages.py` contiene tutti i testi, così si correggono senza toccare la logica; `telegram_channel.py` conosce solo Telegram. Questo porta al primo pattern: il messaggio in arrivo viene trasformato in un `InboundMessage` neutro. Il motore non sa da dove arriva, quindi aggiungere WhatsApp significherebbe scrivere un adattatore, senza toccare il dialogo. Il secondo pattern è l'interfaccia verso il backend, `BackendClient`, con due implementazioni: quella HTTP e quella finta, usata per provare il bot da console senza server. Il terzo è ovviamente la macchina a stati. Sui formati: JSON con token bearer verso il backend, targa normalizzata per la ricerca e, per gli antiduplicati, un requestId con un'impronta SHA-256 della richiesta, che non contiene i dati in chiaro. Se il requestId viene riusato con dati diversi, l'impronta non coincide e il backend rifiuta.

Spiegazione semplice

Ogni file ha un solo mestiere. Puoi cambiare i testi, il canale o il backend separatamente, senza toccare il resto.

File principali

`chatbot/fornitori_chatbot/engine.py` · `chatbot/fornitori_chatbot/messages.py` · `chatbot/fornitori_chatbot/backend.py` · `chatbot/fornitori_chatbot/fake_backend.py` · `chatbot/fornitori_chatbot/__main__.py` · `service/ChatService.java` (`fingerprint`)

Alternative, perché scartate, limiti

Alternative: Scrivere i testi nel codice; bot accoppiato a Telegram; webhook invece del long polling.

Perché: Il long polling evita un indirizzo pubblico (H-009). Il bot risponde solo nelle chat private. La separazione tra canale e motore è documentata nel README del chatbot.

Limiti e gestione: Il dialogo è in RAM. La prova da telefono è segnata come da ripetere (B-10). I 6 test del chatbot coprono le notifiche, non l'intero dialogo.

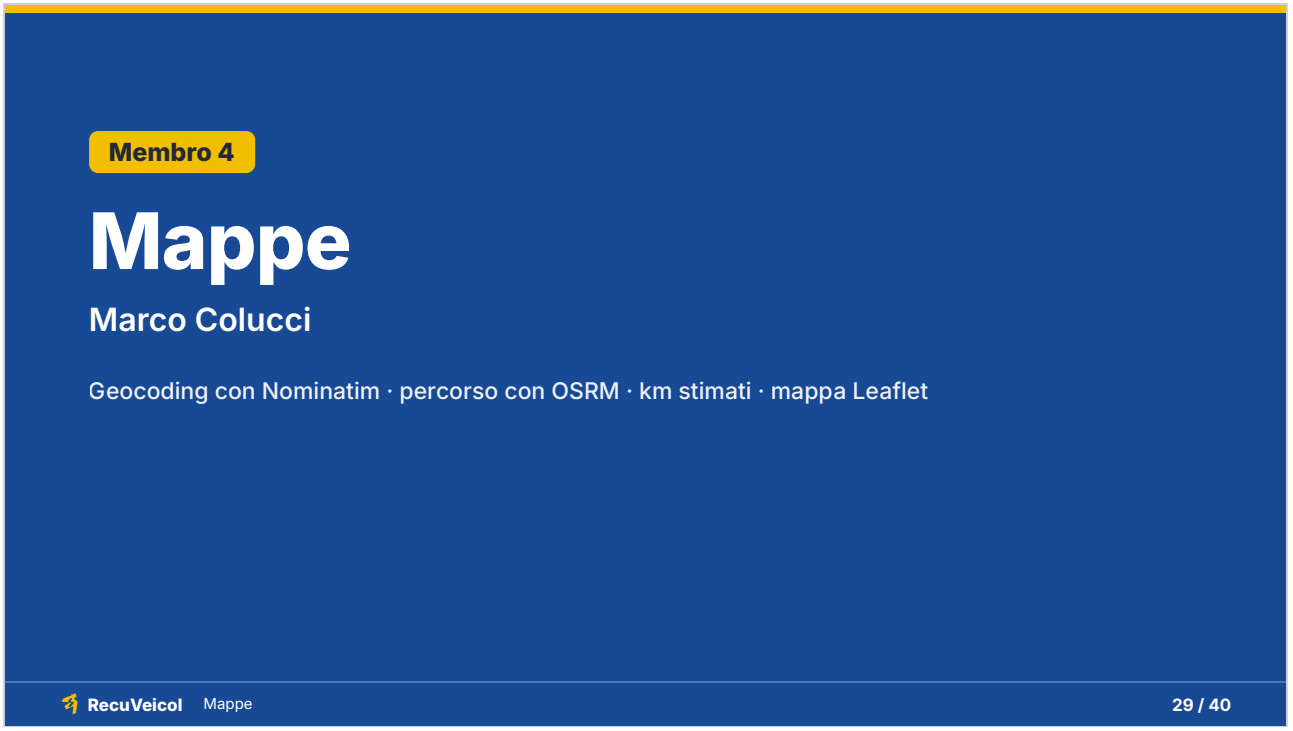
Domande del professore

Perché Python se il backend è Java?

Il bot è separato e parla solo HTTP, quindi può usare un altro linguaggio. Nei documenti non c'è una motivazione formale: il vantaggio verificabile è che gira con la sola libreria standard.

Cosa serve per aggiungere WhatsApp?

Un adattatore che crea `InboundMessage` e una funzione di invio per il dispatcher, oltre a webhook firmati e all'account Business: per questo è fuori ambito.



Cosa dire a voce (60-90 secondi)

Le mappe sono la mia seconda area, e rispondono a tre domande pratiche. Dove si trova il cliente, se ha scritto soltanto un indirizzo? Qual è il percorso dalla base dell'autista al cliente e, per un traino, fino all'officina? Quanti chilometri stimiamo? Per rispondere usiamo due servizi open source e gratuiti: Nominatim trasforma un indirizzo in coordinate, cioè fa il geocoding, e OSRM calcola il percorso su strada. Il risultato viene disegnato nel browser con Leaflet sui tile di OpenStreetMap. Un principio ha guidato questa parte: le mappe aiutano, ma non devono mai bloccare il lavoro. Se un servizio esterno non risponde, l'autista può comunque chiudere la missione inserendo i chilometri. E non inventiamo mai una posizione: se l'indirizzo non si trova, lo diciamo. Vedremo prima il flusso nel backend e poi come appare sullo schermo, con una schermata reale.

Spiegazione semplice

Geocoding: da «Via Bovio, Terlizzi» a latitudine e longitudine. Routing: dal punto A al punto B, seguendo le strade. Leaflet: il disegno sulla pagina.

File principali

`docs/team/MEMBRO_4_MAPS.md` · `controller/MapController.java` · `service/MapService.java` · `docs/management/DECISION_LOG.md` (D-011, D-016, H-010)

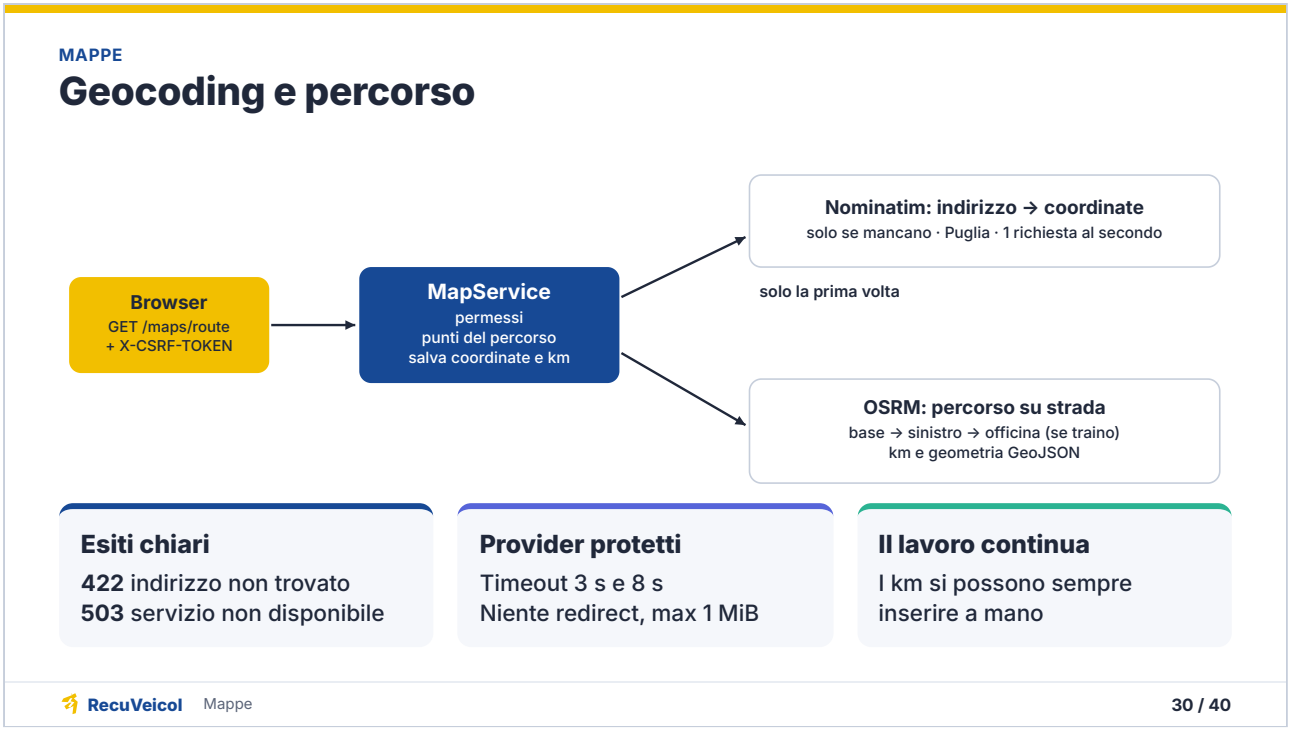
Domande del professore

Che differenza c'è tra km stimati ed effettivi?

Gli stimati vengono da OSRM e sono proposti alla chiusura; gli effettivi li dichiara l'autista e sono quelli usati per il costo.

Quali dati escono dal sistema?

L'indirizzo del sinistro va a Nominatim. Va bene per una demo con dati sintetici, ma è da rivalutare per dati reali.



Cosa dire a voce (60-90 secondi)

Quando si apre una mappa, il browser chiama un'unica API: `maps/route`, indicando la missione e, se serve, l'autista. `MapService` controlla prima i permessi: l'autista vede solo le proprie missioni. Poi compone i punti: la base del mezzo, il luogo del sinistro e, solo per un traino, l'officina. Se il luogo non ha ancora coordinate, chiede a Nominatim di trasformare l'indirizzo e le salva sulla missione, così la conversione avviene una volta sola. La ricerca è limitata alla Puglia e rispetta il limite di una richiesta al secondo. Poi chiede a OSRM il percorso, che torna con i chilometri e la geometria in GeoJSON. I km stimati si salvano solo per l'autista assegnato. Gli errori sono espliciti: 422 se l'indirizzo non esiste, 503 se il servizio è giù. I client hanno timeout di tre e otto secondi, non seguono redirect e leggono al massimo un mebibyte. In nessun caso il lavoro si ferma: i km si inseriscono a mano.

Spiegazione semplice

Il backend fa da intermediario. Il browser chiede «il percorso della missione 31», il backend fa il resto e restituisce un risultato pronto da disegnare.

File principali

`service/MapService.java` · `service/NominatimGeocodingClient.java` · `service/OsrmRoutingClient.java` · `service/MapProviders.java` · `service/ProviderHttp.java` · `dto/MapDtos.java`

Alternative, perché scartate, limiti

Alternative: Google Maps Geocoding e Directions; Mapbox; calcolo diretto dal browser.

Perché: H-010 documenta la scelta: servizi pubblici senza chiavi e costi. Il tramite del backend applica permessi e limiti e può salvare i risultati, mentre la CSP vieta al browser connessioni ad altri domini.

Limiti e gestione: Servizi pubblici senza garanzia di disponibilità (R-04). Fuori dalla Puglia un indirizzo risulta «non trovato» (è configurabile con `fornitori.maps.viewbox`). L'indirizzo va a un servizio esterno.

Domande del professore

Perché non avete usato Google Maps?

Richiede chiave e fatturazione e invia comunque i dati a un terzo. Nominatim e OSRM sono open source e, se serve, si possono ospitare in proprio.

Perché la geocodifica avviene alla prima mappa?

Non serve bloccare la creazione della missione con una chiamata esterna, salvo il traino da chat, dove serve subito per scegliere l'officina.

Perché la GET scrive sul database?

Salva coordinate e km stimati la prima volta. Per questo è protetta dal CSRF; è un'anomalia registrata come V-24.

MAPPE

La mappa sullo schermo: Leaflet



Schermata reale: base a Molfetta, cliente a Terlizzi, officina a Ruvo

- Tile di **OpenStreetMap**, percorso **GeoJSON** di OSRM
- Segnaposto per base, cliente e officina
- Fonte e km stimati sempre indicati
- Autista: si apre da sola. Centrale: su richiesta

15,23 km
stimati da OSRM per questa missione

RecuVeicol Mappe 31 / 40

Cosa dire a voce (60-90 secondi)

Ecco il risultato sullo schermo, ripreso dal sistema in funzione. La missione è un traino: l'autista parte dalla sua base a Molfetta, raggiunge il cliente a Terlizzi e porta l'auto a un'officina a Ruvo di Puglia. La linea viola è la geometria restituita da OSRM. Leaflet la riceve così com'è, in formato GeoJSON, e la disegna sui tile di OpenStreetMap. I tile sono le piccole immagini quadrate che compongono la mappa, e li scarica direttamente il browser: infatti la politica di sicurezza dei contenuti ammette per le immagini quel solo dominio. Sotto la mappa, l'elenco numerato dei punti. Sopra, una frase ricorda la fonte e che i chilometri sono una stima: quelli effettivi si confermano alla chiusura. Nell'area autista la mappa si carica da sola; nella centrale compare premendo «Mostra mappa», scegliendo eventualmente un autista per confrontare il percorso. Senza autista assegnato, si vede solo il punto del cliente.

Spiegazione semplice

Leaflet è il «foglio da disegno» della mappa nel browser. Riceve punti e linea dal backend e non calcola nulla da solo.

File principali

`static/app.js` (`drawMissionMap`) · `static/index.html` (Leaflet da CDN) · `config/SecurityConfig.java` (CSP) · `docs/team/MEMBRO_4_MAPS.md`

Alternative, perché scartate, limiti

Alternative: Google Maps JavaScript API; MapLibre o OpenLayers.

Perché: Leaflet è leggero, gratuito e disegna direttamente il GeoJSON di OSRM, senza chiavi. Il repo documenta Leaflet come scelta (D-011), non il confronto con MapLibre o OpenLayers.

Limiti e gestione: I tile pubblici non sono pensati per un uso intensivo. Serve Internet. Se Leaflet non si carica, la pagina scrive «Mappa non caricata: controlla la connessione».

Domande del professore

Cosa succede se OSRM è giù durante la demo?

La pagina mostra un errore chiaro e l'autista può inserire i km a mano: la chiusura non dipende dalla mappa.

Perché l'autista vede il percorso prima di accettare?

Per valutare la distanza e il guadagno stimato. I dati personali del cliente restano nascosti fino all'accettazione.



Cosa dire a voce (60-90 secondi)

Perché le mappe sono fatte così? Sul fronte file, c'è un solo controller con una sola API, un service che compone il percorso e un client per ciascun provider. `MapProviders` definisce le interfacce `GeocodingClient` e `RoutingClient`: per passare a un altro servizio basta una nuova implementazione, e questo ci permette anche di usare un provider simulato nei test. Il backend fa da intermediario: i permessi sono applicati in un solo punto, e la chiave, se un giorno servisse, non finirebbe mai nel browser. Un pattern importante è che `MapService` non è transazionale, di proposito: i servizi esterni possono metterci secondi e non vogliamo tenere aperta una transazione mentre aspettiamo. I risultati vengono salvati poi, in transazioni brevi. Sui formati: GeoJSON, che è lo standard capito sia da OSRM sia da Leaflet, quindi non serve conversione; coordinate NUMERIC con sei decimali e BigDecimal per i chilometri.

Spiegazione semplice

Il provider si può cambiare senza riscrivere il resto. GeoJSON è un linguaggio comune tra chi calcola il percorso e chi lo disegna.

File principali

`service/MapProviders.java` · `service/MapService.java` · `service/ProviderHttp.java` · `dto/MapDtos.java` · `src/test/java/.../ContractAndProviderTests.java`

Alternative, perché scartate, limiti

Alternative: Chiamare i provider direttamente dentro MissionService; polilinee compresse invece di GeoJSON.

Perché: Separare i client rende il resto indipendente dal provider. `overview=full&geometries=geojson` chiede a OSRM un formato che Leaflet disegna senza decodifica.

Limiti e gestione: Limite di una richiesta al secondo solo per istanza. Il controllo «officina più vicina» e «autista più vicino» usa la linea d'aria, non il percorso su strada.

Domande del professore

Come testate senza chiamare davvero i servizi?

I test usano provider simulati e controllano anche valori non validi, per esempio una distanza negativa (V-20, corretta).

Perché coordinate con sei decimali?

Sei decimali corrispondono a una precisione di circa 10 cm, più che sufficiente per una posizione stradale.



Cosa dire a voce (60-90 secondi)

Siamo Marika Sancilio, Francesco Valeriano e Giuseppe Cangellosi e presentiamo le notifiche. Lo scopo è semplice: avvisare il cliente su Telegram in due momenti, quando un autista prende in carico la richiesta e quando l'intervento è concluso. Il problema vero è l'affidabilità. Che cosa succede se Telegram non risponde, se il bot è spento o se ci sono due bot accesi? Non vogliamo perdere messaggi senza accorgercene e non vogliamo inviarli due volte. La nostra risposta si divide in due parti. Nel backend c'è l'outbox: una coda di notifiche dentro il database, scritta insieme alla missione. Nel bot c'è il dispatcher: ogni cinque secondi riserva le notifiche da inviare, le spedisce e registra l'esito. Vedrete anche come trattiamo il caso limite, che dichiariamo apertamente. Le notifiche partono solo per le missioni create dal bot: per quelle create dalla centrale, la chat del cliente non è nota.

Spiegazione semplice

Le notifiche sono due messaggi al cliente. La parte difficile è inviarli in modo affidabile anche quando qualcosa si rompe.

File principali

`docs/team/MEMBRO_5_NOTIFICATIONS.md` · `entity/NotificationOutbox.java` · `chatbot/fornitori_chatbot/notifications.py` · `database/08_cleanup_and_delivery.sql`

Domande del professore

Perché non partono per le missioni della centrale?

Telegram può scrivere solo a chi ha aperto una chat con il bot, e di quei clienti la chat non è nota. La riga nasce `SKIPPED` e non conserva il telefono.

Il cliente può disattivare gli avvisi?

No: il bot invia solo i due messaggi legati alla sua richiesta. Il consenso non è previsto nei dialoghi del docente (limite dichiarato nel README).



Cosa dire a voce (60-90 secondi)

Il pattern outbox risolve un problema classico. Quando l'autista accetta, dobbiamo fare due cose: aggiornare la missione e avvisare il cliente. Ma il database e Telegram sono due sistemi diversi, e non esiste una transazione che li comprenda entrambi. Se inviassimo prima il messaggio e poi il salvataggio fallisse, il cliente verrebbe avvisato di una cosa mai avvenuta. Se salvassimo e poi Telegram fosse giù, il messaggio andrebbe perso. La soluzione è scrivere la notifica in una tabella dello stesso database, nella stessa transazione della missione. O sono salvate entrambe o nessuna delle due. L'autista riceve subito la risposta, senza aspettare Telegram, e l'invio avviene dopo. La colonna `event_key` è univoca, quindi lo stesso evento non può generare due righe. Il payload contiene l'evento, il codice della missione e il nome dell'autista: nessun dato del cliente. Le missioni della centrale non hanno destinatario, quindi la riga nasce già come SKIPPED.

Spiegazione semplice

Invece di spedire subito, si mette la lettera nella cassetta della posta interna insieme alla modifica. Il postino passerà dopo. La cassetta è nel database, quindi non si perde.

File principali

`service/MissionService.java` (`addOutboxEvent`, `accept`, `complete`) · `entity/NotificationOutbox.java` · `database/01_schema.sql` · `database/08_cleanup_and_delivery.sql` · `docs/diagrams/chat_notifications_sequence.mmd`

Alternative, perché scartate, limiti

Alternative: Invio diretto a Telegram dal service; una coda esterna come RabbitMQ o Kafka.

Perché: D-015 e H-006 documentano l'outbox nella transazione di dominio. Una coda esterna aggiungerebbe un servizio da installare e non risolverebbe da sola l'atomicità con il database. Quest'ultimo confronto è ricostruito: non è nel registro.

Limiti e gestione: Il bot deve controllare periodicamente la coda (ogni 5 secondi), quindi c'è un piccolo ritardo. La tabella cresce: non c'è una politica di pulizia, fuori ambito per la demo.

Domande del professore

Perché non inviare direttamente il messaggio?

Un errore di Telegram bloccherebbe l'accettazione o lascerebbe i dati incoerenti. Con l'outbox i due passaggi sono separati, e il messaggio non si perde.

Perché non Kafka o RabbitMQ?

Sono un'infrastruttura in più da gestire. Avendo già PostgreSQL, una tabella con SKIP LOCKED basta per questi volumi.

Se il bot resta spento per ore?

Le righe restano in `PENDING` e partono alla riaccensione.



Cosa dire a voce (60-90 secondi)

Il dispatcher è la seconda metà. Ogni cinque secondi il bot chiama l'API di claim. Il backend, nella stessa transazione, seleziona fino a venti righe con FOR UPDATE SKIP LOCKED e le passa a SENDING. SKIP LOCKED significa: salta le righe che un altro sta già prendendo. Così due dispatcher non ricevono mai la stessa riga: l'abbiamo provato con quattro in parallelo. Il bot invia e registra l'esito, riportando il numero di tentativo. Se un bot in ritardo manda un esito vecchio, viene ignorato, così non altera una riserva più recente. Se l'invio fallisce, la riga riprova dopo trenta secondi; dopo cinque tentativi passa a FAILED, con l'errore salvato. Se il bot si ferma, la riserva scade dopo cinque minuti e la riga torna disponibile. Resta un solo caso che non possiamo escludere: se il bot si ferma proprio tra l'invio e l'esito, il messaggio può ripartire una volta, perché Telegram non offre una chiave anti-duplicati. Lo dichiariamo come limite accettato per la demo.

Spiegazione semplice

Prima prenoto la lettera, poi la spedisco, poi segno com'è andata. Se sparisco dopo aver prenotato, dopo cinque minuti qualcun altro può riprenderla.

File principali

`chatbot/fornitori_chatbot/notifications.py` · `repository/NotificationOutboxRepository.java` (`lockClaimable`) · `service/ChatService.java` (`claimNotifications`, `reportNotification`) · `src/test/java/.../ChatAndNotificationTests.java` · `chatbot/tests/test_notifications.py`

Alternative, perché scartate, limiti

Alternative: Consegna «esattamente una volta»; gestire la coda senza stato SENDING.

Perché: L'esattamente una volta non è ottenibile senza una chiave anti-duplicati del provider: per questo D-015 dichiara «almeno una volta in caso di crash». La riserva persistente è documentata in D-015 e nel contratto d'integrazione.

Limiti e gestione: V-14, accettata per la demo: messaggio duplicato possibile in caso di crash. Nel processo vivo, il dispatcher ricorda le notifiche inviate con esito non registrato e non le rispedisce. FAILED non è inviata: va letta su `ultimo_errore`.

Domande del professore

Quindi i duplicati sono impossibili?

Tra dispatcher sì. Resta possibile un doppio invio se il bot si ferma tra invio ed esito: è un compromesso consapevole, preferibile a perdere il messaggio.

Perché «almeno una volta» e non «al massimo una volta»?

Meglio un avviso ripetuto che un cliente che non sa che l'autista sta arrivando.

Chi controlla le notifiche in FAILED?

Oggi nessuna interfaccia le mostra: si leggono su `notifications_outbox.ultimo_errore`. È un possibile miglioramento.



Cosa dire a voce (60-90 secondi)

Ricapitoliamo perché abbiamo strutturato così le notifiche. I file seguono la divisione delle responsabilità: nel backend, l'entità `NotificationOutbox` e la logica di claim ed esito; nel bot, soltanto `notifications.py`. Il bot non tocca il database: passa da due API dedicate. I pattern sono tre. Il transactional outbox garantisce che evento e dato nascano insieme. La riserva con scadenza, che in inglese si chiama lease, evita che un bot caduto blocchi per sempre la coda. E il dispatcher è indipendente dal canale: riceve una funzione di invio e non sa nulla di Telegram, quindi un nuovo canale richiede solo un adattatore. Sui formati: il payload è in JSONB e contiene il minimo, per non duplicare dati personali in una tabella di servizio. Gli stati sono controllati da un vincolo CHECK nel database. Infine, il numero di tentativo lega l'esito alla riserva corretta. Tutto questo è testato anche con dispatcher concorrenti.

Spiegazione semplice

Il database è il punto di verità anche per le notifiche. Il bot è un semplice esecutore che può fermarsi e ripartire senza danni.

File principali

`entity/NotificationOutbox.java` · `chatbot/fornitori_chatbot/notifications.py` · `controller/ChatController.java` · `database/09_integrity_and_indexes.sql` · `docs/team/MEMBRO_5_NOTIFICATIONS.md`

Alternative, perché scartate, limiti

Alternative: Dispatcher dentro il backend Java.

Perché: Il bot è già il processo che parla con Telegram e possiede il token: tenere lì il dispatcher evita di dare il token al backend. Il repository descrive la soluzione ma non questo confronto, quindi il motivo è ricostruito.

Limiti e gestione: Il dispatcher parte solo con `--telegram`. Se il processo del bot è fermo, non parte nessun invio, anche se il backend funziona.

Domande del professore

Perché il dispatcher sta nel bot?

Così il token Telegram resta solo nel bot, e il backend non deve conoscere il provider.

Come aggiungereste gli SMS?

Con un nuovo canale e una funzione di invio. I valori ammessi in `channel` vanno estesi: oggi sono `telegram`, `whatsapp` e `internal`.



Cosa dire a voce (60-90 secondi)

Parliamo apertamente di sicurezza e limiti. Abbiamo fatto un audit del backend e tenuto un registro con 25 voci: 16 sono corrette, due sono accettate per la demo e sette restano aperte. Tra i controlli attivi ci sono BCrypt, il CSRF, i ruoli verificati dal server, i limiti ai tentativi, le intestazioni di sicurezza e 139 test su database isolato. Il limite principale è la voce V-01, ad alta gravità: la sola targa identifica il cliente. Chi conosce una targa può vedere nome e compagnia e creare una richiesta. È coerente con i dialoghi del docente, ma va bene solo con dati sintetici. Nell'audit è già descritta la soluzione: un secondo dato, come le ultime cifre della polizza. WhatsApp è fuori ambito, e usiamo Telegram. HTTPS non è stato provato su un server reale. Il dialogo sta in memoria e i provider delle mappe non offrono garanzie. La conclusione è chiara: è una demo robusta, ma non è pronta per dati reali.

Spiegazione semplice

Sapere che cosa non è pronto è parte del lavoro. Il registro dice cosa è stato corretto, cosa è stato accettato e cosa resta aperto, con il motivo.

File principali

`docs/security/VULNERABILITY_REGISTER.md` · `docs/security/BACKEND_SECURITY_AUDIT.md` · `docs/security/SECURITY_TEST_REPORT.md` · `docs/architecture/SECURITY_DESIGN.md` · `docs/management/RISK_REGISTER.md` · `docs/word/AUDIT_SICUREZZA.docx`

Alternative, perché scartate, limiti

Alternative: Aggiungere già ora un secondo fattore alla chat.

Perché: Il PDF del docente chiede solo di verificare che la targa corrisponda a un assicurato: cambiare il dialogo si allontanerebbe dai materiali. L'audit propone di introdurlo dietro una proprietà, da concordare.

Limiti e gestione: Voci aperte: V-01 sola targa; V-13 ruolo DB proprietario; V-15 HTTPS non provato; V-21 token statico; V-23 liste non paginate; V-24 GET delle mappe che scrive; V-25 nessun limite alle richieste aperte. Accettate: V-14 possibile doppio messaggio; V-22 limiti in memoria. Non verificati: Telegram reale nel rapporto di test, Windows, vulnerabilità delle dipendenze.

Domande del professore

Avete fatto un penetration test?

No: l'audit è una verifica applicativa sui casi descritti e non una certificazione. Lo dice il documento stesso.

Qual è la cosa più grave?

V-01: la targa rivela nome e compagnia. La soluzione è pronta sulla carta, ma va concordata con lei perché cambia il dialogo.

Il token del bot è al sicuro?

Sta solo nelle variabili d'ambiente. È però statico, senza rotazione automatica (V-21).

CHIUSURA

Scaletta della demo live

#	Dove	Cosa mostriamo
1	Sito pubblico	RecuVeicol e il collegamento al bot
2	Telegram	Targa, luogo, traino, problema: nasce la richiesta
3	Centrale	Scheda missione, **Proponi al più vicino**, conto alla rovescia
4	Autista (telefono)	Proposta e mappa, **Accetta**: compaiono i contatti
5	Telegram	Notifica automatica con il nome dell'autista, poi **STATO**
6	Autista	Chiusura con i km, costo calcolato dal server
7	Telegram e centrale	Notifica di conclusione, dashboard aggiornata

Se Telegram non risponde: bot da console sul backend reale. **Se le mappe non rispondono:** km inseriti a mano.

RecuVeicol Chiusura 38 / 40

Cosa dire a voce (60-90 secondi)

Adesso vi facciamo vedere il sistema. Seguiremo una missione per intero, in sette passi. Partiamo dal sito pubblico, che porta al bot. Su Telegram scriviamo la targa di un cliente dimostrativo, il luogo, scegliamo il traino e descriviamo il problema: nasce la richiesta. Passiamo alla centrale: la scheda è già lì, premiamo «Proponi al più vicino» e parte il conto alla rovescia. Poi l'autista, dal telefono, vede la proposta e la mappa, ma non i contatti del cliente; accetta, e i contatti compaiono. Torniamo su Telegram: entro pochi secondi arriva la notifica con il nome dell'autista, e con STATO vediamo la frase aggiornata. L'autista chiude la missione con i chilometri e il server calcola il costo. Infine arriva l'ultimo messaggio e la dashboard si aggiorna. Abbiamo anche un piano di riserva: se Telegram non risponde, usiamo il bot da console sul backend reale; se le mappe non rispondono, i km si inseriscono a mano.

Spiegazione semplice

È il copione della demo. Tenere aperte tre finestre (centrale, autista, Telegram) ed eseguire i passi in ordine. Le notifiche richiedono il bot acceso con `--telegram`.

File principali

`docs/manuals/MANUALE_UTENTE.md` · `chatbot/README.md` · `README.md` · `database/02_seed.sql` (targa d'esempio PW 660 SW)

Alternative, perché scartate, limiti

Alternative: Video registrato al posto della demo dal vivo.

Perché: La demo dal vivo dimostra che il sistema funziona davvero. Le schermate di queste slide restano utilizzabili se qualcosa non si avvia.

Limiti e gestione: Servono Internet (Bootstrap, Leaflet, tile, Nominatim, OSRM, Telegram), database con script applicati fino al 10 e credenziali demo prese dai materiali del docente. Una missione di notte applica il +25%, una di giorno no.

Domande del professore

Cosa succede se Telegram non risponde?

Mostriamo il dialogo dalla console con `--http`: il motore e il backend sono gli stessi, cambia solo il canale.

Posso provare io con una targa?

Sì, purché presente nel seed e con polizza valida; le targhe non trovate vengono rifiutate e contate nel limite.



Cosa dire a voce (60-90 secondi)

Che cosa faremmo dopo. La prima priorità è l'identità del cliente: chiedere un secondo dato oppure un codice SMS, chiudendo la voce V-01. Poi i canali: WhatsApp e i webhook in ingresso sono già descritti nel contratto OpenAPI, come operazioni progettate e non implementate, e il motore del bot è già separato dal canale. La terza è la gestione utenti dall'interfaccia, oggi fuori ambito perché un autista richiede anche un mezzo dedicato. Poi la messa in produzione: HTTPS provato davvero, un ruolo del database per le migrazioni e uno per l'applicazione, log centralizzati e backup. Infine la qualità del codice: liste paginate e un `MissionService` diviso in parti più piccole. A destra ci sono le decisioni che aspettano una conferma da parte sua, perché cambiano il comportamento: la fascia notturna, l'ora di riferimento della tariffa, il canale e il livello di identificazione richiesto. Alcune si modificano in pochi minuti.

Spiegazione semplice

Tutti gli sviluppi vengono dai registri del progetto: non sono promesse generiche. Le quattro decisioni a destra dipendono dal docente.

File principali

`docs/security/BACKEND_SECURITY_AUDIT.md` (soluzione per V-01) · `api/openapi.yaml` (operazioni `designed`) · `docs/agile/PRODUCT_BACKLOG.md` · `docs/management/DECISION_LOG.md` (decisioni da richiedere) · `docs/architecture/TAD_FORNITORI.md` (§11)

Alternative, perché scartate, limiti

Alternative: Aggiungere altre funzioni prima di chiudere i limiti noti.

Perché: Prima le voci che impediscono l'uso con dati reali: R-01, R-06, R-12 e R-13 sono indicate come da risolvere per dichiarare l'idoneità.

Limiti e gestione: Nessuno di questi sviluppi è implementato. Il dialogo 10 con Gemini è facoltativo e non è nei piani obbligatori.

Domande del professore

Quanto ci vuole per cambiare la fascia notturna?

Poco: i valori sono in `pricing_config`. Lo storico non viene ricalcolato: resta il costo salvato.

Da dove inizieresti per andare in produzione?

Dal secondo dato per l'identità e da HTTPS: sono i requisiti minimi prima di trattare dati reali.



Cosa dire a voce (60-90 secondi)

Per concludere. Abbiamo costruito un sistema completo, che porta una richiesta di soccorso dal primo messaggio del cliente fino alla chiusura con il costo calcolato. Tutte le regole, i permessi e i costi stanno sul server: il browser e il bot non decidono nulla. I componenti sono separati e collegati da un unico contratto, così ognuno può evolvere senza rompere gli altri. Le notifiche nascono nella stessa transazione della missione e si possono ricostruire in ogni momento. E soprattutto conosciamo i nostri limiti: sono scritti in un registro e li abbiamo dichiarati qui. Come team abbiamo imparato a lavorare su un contratto condiviso, a scegliere tecnologie con un motivo e a difendere le scelte con i test. Vi ringraziamo per l'attenzione e siamo a disposizione per le domande. Ognuno di noi può rispondere sulla propria parte, e abbiamo aperti il codice e i documenti, se volete vedere qualcosa nel dettaglio.

Spiegazione semplice

Il messaggio finale in cinque punti. Se la domanda riguarda una parte precisa, risponde il responsabile di quella parte: sotto trovi le domande probabili, nell'appendice.

File principali

`README.md` · `docs/00_INDICE_GENERALE.md` · `docs/team/README.md`

Domande del professore

Qual è stata la parte più difficile?

Da concordare nel team, in base all'esperienza di ciascuno. Tecnicamente i punti più delicati sono concorrenza delle assegnazioni e affidabilità delle notifiche.

Il progetto è terminato?

Il repository indica lo Sprint 4 in chiusura, con l'accettazione del docente da acquisire e la prova Telegram da telefono da ripetere.

Appendice · Glossario

Solo nella guida di studio. I termini tecnici in parole semplici.

API REST
Modo di offrire funzioni tramite HTTP e JSON: ogni operazione ha un indirizzo e un verbo (GET, POST, PATCH).

Audit (tabella eventi)
Registro delle azioni avvenute: `mission_events` conserva chi ha fatto cosa e quando.

BCrypt
Funzione per salvare le password come hash lento, non reversibile.

BigDecimal
Tipo Java per numeri decimali esatti, usato per i soldi.

Bootstrap
Libreria CSS e JavaScript con griglia, modali e tabelle pronte.

CDN
Rete di server pubblici da cui il browser scarica librerie come Bootstrap e Leaflet.

CI (GitHub Actions)
Ripete in automatico build e test a ogni modifica.

Claim (riserva)
Il dispatcher prenota una notifica, che non può essere presa da altri per 5 minuti.

Controller
Classe che riceve la richiesta HTTP, valida l'input e chiama il service.

CSP
Content-Security-Policy: dice al browser da quali siti può caricare script, immagini e connessioni.

CSRF
Attacco in cui un altro sito invia richieste con il tuo cookie. Si blocca con un token da allegare.

Dispatcher
Parte del bot che preleva le notifiche dalla coda, le invia e registra l'esito.

DTO
Oggetto che descrive i dati in entrata e in uscita dalle API, separato dalla tabella.

Entity
Classe Java che rappresenta una riga di una tabella.

Fixture
Dato storico importato dal seed: `is_fixture=TRUE`, mai modificato dalle azioni live.

Geocoding
Trasformare un indirizzo in coordinate. Lo fa Nominatim.

GeoJSON
Formato JSON per punti e linee geografiche: lo produce OSRM e lo disegna Leaflet.

Haversine
Formula della distanza in linea d'aria tra due coordinate (`GeoDistance`).

Hibernate / JPA
Strumenti che collegano classi Java e tabelle. In `validate` controllano lo schema senza modificarlo.

HttpOnly
Attributo del cookie: JavaScript non lo può leggere.

Idempotenza
Ripetere la stessa richiesta non produce un secondo effetto: lo garantisce il `requestId`.

Indice parziale
Indice valido solo per le righe che rispettano una condizione: `uq_driver_active_live`.

Leaflet
Libreria JavaScript open source per mappe interattive.

Lock pessimistico
La riga viene bloccata finché la transazione non finisce: chi arriva dopo attende.

Long polling
Il bot chiede a Telegram se ci sono messaggi, tenendo aperta la richiesta: non serve un indirizzo pubblico.

Maven Wrapper
Script `mvnw` che usa la versione di Maven indicata nel repository.

Nominatim
Servizio open source di geocoding basato su OpenStreetMap.

OpenAPI
Formato standard che descrive le API: nel progetto è `api/openapi.yaml`.

OSRM
Servizio open source che calcola percorsi su strada.

Outbox
Tabella-coda: l'evento viene scritto nella stessa transazione del dato e inviato dopo.

Repository
Interfaccia Spring Data che esegue le query su una tabella.

Scheduler
Attività ripetuta in automatico: `MissionExpiryScheduler` controlla le scadenze ogni 30 secondi.

Seed
Dati iniziali di esempio, in `database/02_seed.sql`.

Service
Classe con le regole di business e le transazioni.

SKIP LOCKED
Clausola SQL che salta le righe già bloccate da altri: usata per la coda delle notifiche.

Tile
Tassello d'immagine che compone la mappa.

Transazione
Gruppo di operazioni salvate tutte insieme o nessuna.

Webhook
Il servizio esterno chiama un nostro indirizzo pubblico. Qui è fuori ambito: si usa il long polling.

Appendice · Domande probabili del professore

Risposte brevi, per membro.

Membro 1 · Backend e database · Vincenzo Minervini

Perché Spring Boot e non Node.js?

Spring porta già sessione, CSRF, validazione e JPA con lock pessimistici. Il repo non contiene un confronto formale: il motivo documentato è lo stack condiviso (D-001).

Perché PostgreSQL e non MySQL?

Il vincolo «un autista, una missione attiva» è un indice parziale con WHERE, che MySQL non supporta in questa forma.

Come evitate la doppia assegnazione?

Lock su missione e autista nel service, e indice unico `uq_driver_active_live` nel database. Il secondo riceve 409.

Quando scade una proposta?

Dopo 10 minuti (`pricing_config`). Lo scheduler controlla ogni 30 secondi e il service ricontrolla alla risposta.

Come si calcola il costo?

50 + 1,20 × km effettivi; +25% se l'ora di accettazione è tra le 22:00 e le 08:00, Europe/Rome, arrotondamento HALF_UP. Es.: 15,23 km = €68,28 di giorno, €85,35 di notte.

Perché 22:00 se il PDF dice 20:00?

Scelta del team (D-HARD-03), da confermare col docente. Nello storico 6 missioni su 30 sono notturne; 3 lo sono solo con la vecchia fascia e non vengono ricalcolate.

Che cos'è is_fixture?

Segna le 30 missioni storiche: lo scheduler e le azioni live le ignorano, così i totali di riferimento restano stabili.

Come testate?

139 test Java su database `fornitori_e2e*`, che partono solo con `FORNITORI_TEST_DATABASE=isolated`. La pipeline li esegue in CI.

Perché le GET delle mappe usano il CSRF?

Perché possono salvare coordinate e km stimati (V-24).

Membro 2 · Frontend · F. Valeriano, G. Cangellosi, M. Sancilio

Perché niente React?

Poche schermate, nessuna build, nessun Node.js da installare. Il motivo è ricostruito: il repo registra la scelta (D-010) senza confronto.

Come parla il frontend col backend?

Con `fetch` verso API sulla stessa origine, cookie di sessione e header `X-CSRF-TOKEN` sulle modifiche.

Come si aggiorna la pagina?

Con un controllo ogni 15 secondi su avvisi e proposte; l'area autista si ridisegna solo se qualcosa è cambiato.

Perché l'autista non vede subito i contatti?

È una regola del server: nome e telefono arrivano solo dopo l'accettazione.

Come proteggete dall'XSS?

Escaping con `safe` su tutti i dati dinamici e Content-Security-Policy.

Funziona senza Internet?

No: Bootstrap, Leaflet e i tile arrivano da servizi esterni.

Membro 3 · Chatbot · Marco Colucci

Perché Telegram e non WhatsApp?

API gratuita e aperta, nessuna approvazione Meta, adatta alla demo; il long polling evita un indirizzo pubblico.

Perché Python?

Separato dal backend, parla solo HTTP e usa la sola libreria standard. Il repo non motiva formalmente la scelta del linguaggio.

Perché il bot non legge il database?

Requisito NFR-005: le regole restano nel backend, che le applica una volta sola.

Come evitate una missione doppia?

`requestId` uguale nei tentativi e impronta SHA-256: stesso invio, stesso risultato; dati diversi, 409.

La targa basta a identificare?

No: è il limite V-01. Va bene solo con dati sintetici; la soluzione è nell'audit.

Cosa succede dopo un riavvio?

I dialoghi in corso si perdono, le missioni restano nel backend.

Perché nessuna AI?

Il flusso guidato è prevedibile (D-005). Il dialogo 10 con Gemini è facoltativo e non implementato.

Membro 4 · Mappe · Marco Colucci

Perché non Google Maps?

Richiede chiave e fatturazione. H-010: servizi pubblici senza chiavi e costi nel perimetro didattico.

Cosa fa Nominatim, cosa fa OSRM?

Nominatim: indirizzo in coordinate. OSRM: percorso, km e geometria.

Perché la chiamata passa dal backend?

Permessi e limiti in un solo punto, salvataggio dei risultati, e la CSP vieta al browser connessioni ad altri domini.

Km stimati o effettivi?

Gli stimati sono proposti alla chiusura; il costo usa quelli effettivi.

Se OSRM è giù?

Errore 503 chiaro; l'autista inserisce i km a mano.

Perché il più vicino in linea d'aria?

Non richiede una chiamata esterna per ogni autista. Può non essere il migliore su strada: la centrale può scegliere a mano.

Membro 5 · Notifiche · M. Sancilio, F. Valeriano, G. Cangellosi

Che cos'è il pattern outbox?

L'evento viene scritto in una tabella nella stessa transazione della missione e inviato in seguito da un altro processo.

Perché non inviare direttamente?

Database e Telegram non condividono una transazione: un guasto produrrebbe dati incoerenti o messaggi persi.

Come evitate i doppi invii?

`FOR UPDATE SKIP LOCKED` tra dispatcher; `event_key` univoca; numero di tentativo nell'esito.

I doppi invii sono impossibili?

No: un crash tra invio ed esito può ripetere il messaggio una volta (V-14, accettata per la demo).

Quando una notifica fallisce?

Riprova dopo 30 secondi; dopo 5 tentativi va in `FAILED` con l'errore salvato.

Perché le missioni della centrale non notificano?

La chat del cliente non è nota: la riga nasce `SKIPPED`, senza telefono.

Appendice · Alternative considerate

La colonna «Base» dice da dove viene il motivo. «Documentata»: nel registro decisioni o nei manuali. «Dal codice»: si vede da ciò che il codice usa. «Ricostruita» o «Non documentata»: non c'è un confronto scritto nel repository, quindi va presentata come valutazione del team, dopo averla confermata.

Area	Scelta	Scartata	Motivo	Base
Backend	Java 25 + Spring Boot	Node.js + Express	Sessione, CSRF, validazione e JPA già integrati. Con Node andrebbero scelte e combinate più librerie.	Ricostruita dal codice
Build	Maven Wrapper	Gradle	Stessa versione per tutti con `mvnw`. Nessun confronto con Gradle nel repo.	Non documentata
Database	PostgreSQL 18	MySQL	Indici parziali con WHERE (`uq_driver_active_live`, `uq_one_open_proposal`) non disponibili in questa forma.	Dal codice
Database	Relazionale	NoSQL (MongoDB)	Relazioni e vincoli forti tra missioni, clienti e autisti; transazioni tra missione e outbox.	Ricostruita dal codice
Schema	Script SQL numerati	ddl-auto di Hibernate	Modifiche esplicite e rileggibili; `validate` segnala gli script mancanti.	Documentata (README, TAD)
Frontend	HTML, CSS, JavaScript	React	Niente build e niente Node (manuale tecnico). Il confronto con React non è registrato.	Ricostruita dal codice
Mappa	Leaflet + OSM	Google Maps JS	Niente chiavi né fatturazione; disegna direttamente il GeoJSON.	Documentata (H-010, D-011)
Percorsi	Nominatim e OSRM	Google Directions e Geocoding	Open source, senza chiavi né costi nel perimetro didattico.	Documentata (H-010)
Canale	Telegram	WhatsApp Business API	API gratuita e aperta, nessuna approvazione Meta, adatta alla demo. WhatsApp è fuori ambito (D-009).	Brief del team; D-009 per l'ambito
Ingresso bot	Long polling	Webhook	Non serve un indirizzo pubblico né ngrok.	Documentata (H-009)
Libreria bot	Sola libreria standard	python-telegram-bot	Nessuna dipendenza da installare. Nessun confronto registrato.	Dal codice
Linguaggio bot	Python	Java nel backend	Bot indipendente, solo HTTP. Motivazione formale assente.	Non documentata
Dialogo	Flusso a stati	AI / Gemini	Prevedibile e verificabile. Il dialogo con Gemini è facoltativo.	Documentata (D-005)
Notifiche	Outbox su PostgreSQL	Invio diretto o Kafka/RabbitMQ	Atomicità con la missione, nessun servizio in più.	Outbox documentato (D-015); confronto ricostruito
Autenticazione	Sessione + CSRF	JWT	Stessa origine, revoca immediata al logout, cookie HttpOnly.	Documentata (H-004)
Denaro	BigDecimal, NUMERIC	double, FLOAT	Niente errori di arrotondamento; HALF_UP.	Documentata (H-003)
Architettura	Monolite a livelli	Microservizi	Conservare monolite, livelli e API condivise.	Documentata (D-HARD-01)